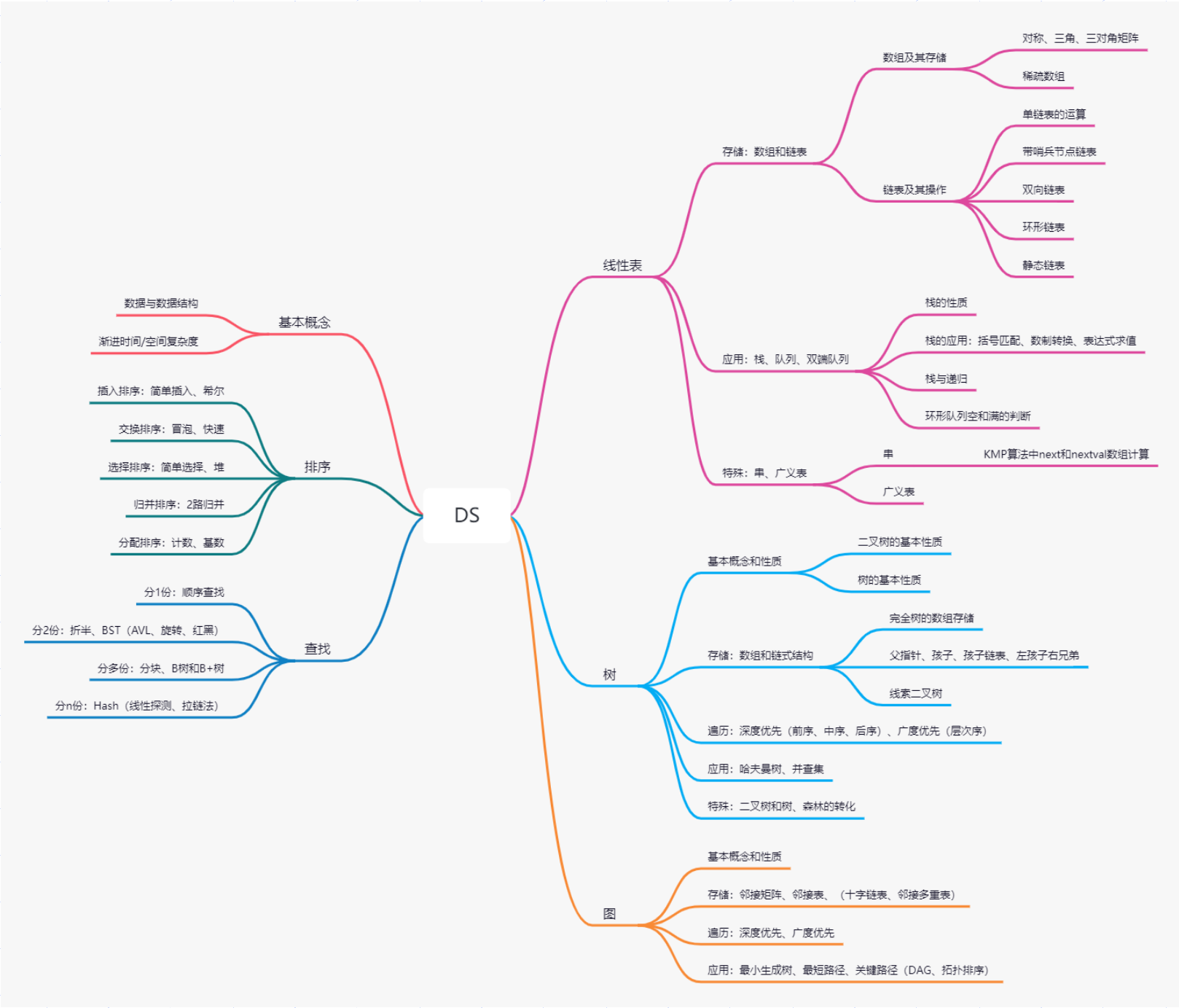
Data Structure

复旦大学软件学院

DS概览


DS
基本概念
数据与数据结构
渐进时间/空间复杂度
排序
插入排序：简单插入、希尔
交换排序：冒泡、快速
选择排序：简单选择、堆
归并排序：2路归并
分配排序：计数、基数
查找
分1份：顺序查找
分2份：折半、BST（AVL、旋转、红黑）
分多份：分块、B树和B+树
分n份：Hash（线性探测、拉链法）
线性表
存储：数组和链表
数组及其存储
对称、三角、三对角矩阵
稀疏数组
链表及其操作
单链表的运算
带哨兵节点链表
双向链表
环形链表
静态链表
应用：栈、队列、双端队列
栈的性质
栈的应用：括号匹配、数制转换、表达式求值
栈与递归
环形队列空和满的判断
特殊：串、广义表
串
KMP算法中next和nextval数组计算
广义表
树
基本概念和性质
二叉树的基本性质
树的基本性质
存储：数组和链式结构
完全树的数组存储
父指针、孩子、孩子链表、左孩子右兄弟
线索二叉树
遍历：深度优先（前序、中序、后序）、广度优先（层次序）
应用：哈夫曼树、并查集
特殊：二叉树和树、森林的转化
图
基本概念和性质
存储：邻接矩阵、邻接表、（十字链表、邻接多重表）
遍历：深度优先、广度优先
应用：最小生成树、最短路径、关键路径（DAG、拓扑排序）

重点难点

- 概念
  - 时间复杂度

- 线性表
  - 链表操作
  - 栈的概念和应用
  - 环形队列
  - 特殊矩阵存储
  - 串的匹配
  - 广义表*

重点难点

- 树
  - 二叉树的基本概念和性质
  - 二叉树的遍历
  - 哈夫曼树
- 图
  - 图的基本概念和存储
  - 最小生成树
  - Dijkstra算法及代码
  - 关键路径

重点难点

- 查找
  - AVL树
  - B树
  - Hash
- 排序
  - 快速排序
  - 堆排序
  - 排序的比较

真题复习

时间复杂度

- 下列程序的时间复杂度是（）

```
count = 0;
for(k=1;k<=n;k*=2)
    for(j=1;j<=n;j++)
        count++;
```

A. O(log n)
B. O(n)
C. O(n log n)
D. $O(n^2)$

时间复杂度

- 下列程序的时间复杂度是（）

```
void fun(int n){
    int i=0;
    while(i*i*i<=n)
        i++;      }
```

A. O(n)
B. O(log n)
C. O(n^1/3)
D. O(n^1/2)

时间复杂度

- 下列程序的时间复杂度是（）

```
x = 0;
while(n>=(x+1)*(x+1))
  x = x+1;
```

A. O(n)
B. O(log n)
C. O(n^2)
D. O(n^1/2)

链表与时间复杂度

- 将长度为n的单链表链接在长度为m的单链表后面，其算法复杂度采用大O形式表示应该是（）
  A. O(1)
  B. O(m)
  C. O(n)
  D. O(m+n)

链表与时间复杂度

- 已知两个长度分别为m和n的升序链表，若将他们合并为长度为m+n的一个降序链表，则最坏情况下的时间复杂度为（）
  A. O(n)
  B. O(mn)
  C. O(min(m,n))
  D. O(max(m,n))

链表与时间复杂度

- 已知单链表L，查找链表中倒数第k个位置上节点代码的最优时间复杂度是（）。
  A. O(n)
  B. O(log n)
  C. O(n^2)
  D. O(k)

双向链表

- 已知带有头结点的双向链表L，结点结构为 

现在要删除指针p所指的结点，可能要用到的语句是（）。

1. p->next->prev = p->prev;
2. p->next->prev = p->next;
3. p->prev->next = p->prev;
4. p->prev->next = p->next;

A. 1, 3　　B. 1, 4　　C. 2, 3　　D. 2, 4

栈

- 若元素a、b、c、d、e、f依次进栈，允许进栈、出栈操作交替进行，但不允许连续3次进行退栈操作，则不可能得到的出栈序列是（）
  - A. dcebfa
  - B. cbdaef
  - C. bcaefd
  - D. afedcb

栈

- 元素a, b, c, d, e, f依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直至所有元素都出栈，则在所有可能的出栈序列中，以元素d e开头的序列个数是（）
  - A. 3
  - B. 4
  - C. 5
  - D. 6

栈

- 一个栈的入栈序列为1，2，...，n，出栈序列为P1，P2，...，Pn。若P2=3，则P3可能取值的个数是（）。

A. n-3
B. n-2
C. n-1
D. 无法确定

栈

- 给定有限符号集S，in和out均为S中所有元素的任一排列，对于初始为空的栈ST，下列叙述中，正确的是（）。
  A. 若in是ST的入栈序列，则不能判断out是否为其可能的出栈序列
  B. 若out是ST的出栈序列，则不能判断in是否为其可能的入栈序列
  C. 若in是ST的入栈序列，out是对应in的出栈序列，则in与out一定不同
  D. 若in是ST的入栈序列，out是对应in的出栈序列，则in与out可能互为倒序

栈

- 现有一个共享栈S，其低位是栈S1，高位是栈S2，低位栈的栈顶地址从低向高增长，高位栈的栈顶地址从高到低减小，要求该共享栈在一端非空时，栈指针指向当前元素的下一位置，则在下列说法中，（）是错误的。

A. 该共享栈降低了溢出的可能性，提高了空间利用率

B. 当两端都非空且S1.top+1=S2.top时，共享栈满

C. S1的入栈操作是S[S1.top++]=x，出栈操作是x=S[--S1.top]

D. S2的入栈操作是S[S2.top--]=x，出栈操作是x=S[++S2.top]

栈的应用

- 如果有一组双引号序列（如“”“”“””“）需要验证其是否正确匹配，可能要用到的数据结构是（）。
  - A. 计数器
  - B. 栈
  - C. 队列
  - D. 链表

栈的应用

- 假设栈初始为空，将中缀表达式a/b+(c*d-e*f)/g转化为等价的后缀表达式的过程中，当扫描到f时，栈中的元素依次是（）。
  A. +(*-
  B. +(-*
  C. /+(*-*
  D. /+-*

队列

- 设栈S和队列Q的初始状态均为空，元素abcdefg依次入栈S。若每个元素出栈后立刻入队列Q，且7个元素出队的顺序是bdcfeag，则栈的容量至少是（）。
  - A. 1
  - B. 2
  - C. 3
  - D. 4

队列

- 循环队列放在一维数组A[0...M-1]中，end1指向队头元素，end2指向队尾元素后一个位置。假设队列两端均可进行出队和入队操作，队列中最多能容纳M-1个元素。初始时为空。下列判断队空和队满条件中，正确的是（）。

A. 队空：end1 == end2；队满：end1==(end2+1) mod M

B. 队空：end1 == end2；队满：end2==(end1+1) mod (M-1)

C. 队空：end2==(end1+1) mod M；队满：end1==(end2+1) mod M

D. 队空：end1==(end2+1) mod M；队满：end2==(end1+1) mod (M-1)

矩阵的压缩存储

- 设有一个12×12的对称矩阵M，将其上三角部分的元素$m_{i,j}$（1<=i<=j<=12）按行优先存入C语言的一维数组N中，元素$m_{6,6}$在N中的下标是（）。
  - A. 50
  - B. 51
  - C. 55
  - D. 66

矩阵的压缩存储

- 设有一个10×10的对称矩阵M，将其上三角部分的元素$m_{i,j}$（1<=i<=j<=10）按列优先存入C语言的一维数组N中，元素$m_{7,2}$在N中的下标是（）。
  A. 15
  B. 16
  C. 22
  D. 23

矩阵的压缩存储

- 有一个100阶的三对角矩阵M，其元素$m_{i,j}$（1<=i,j<=100）按行优先依次压缩存入下标从0开始的一维数组N中。元素$m_{30,30}$在N中的下标是（）。
  - A. 86
  - B. 87
  - C. 88
  - D. 89

稀疏矩阵

- 若采用三元组表存储结构存储稀疏矩阵M，则除三元组表外，下列数据中还需要保存的是（）。

I. M的行数　　II. M中包含非零元素的行数

III. M的列数　　IV. M中包含非零元素的列数

A. 仅I, III　　B. 仅I, IV　　C. 仅II, IV　　D. I, II, III, IV

KMP算法

- 已知字符串S为"abaabaabacacaabaabcc"，模式串t为"abaabc"。采用KMP算法进行匹配，第一次出现失配（s[i]≠t[j]）时，i=j=5，则下次开始匹配时i和j的值分别是（）。

A. i=1,j=0
B. i=5,j=0
C. i=5,j=2
D. i=6,j=2

KMP算法

- 已知字符串S为"abaabaabcabaabc"，模式串t为"abaabc"。采用KMP算法进行匹配，到匹配成功时为止，在匹配过程中进行的单个字符间比较次数是（）。
  - A. 9
  - B. 10
  - C. 12
  - D. 15

树

- 在一棵度为4的树T中，若有20个度为4的节点，10个度为3的节点，1个度为2的节点，10个度为1的节点，则树T的叶子节点个数为（）。

A. 41
B. 82
C. 113
D. 122

树

- 若三叉树T中有244个结点，叶结点高度为1，则T的高度至少为（）。

A. 8
B. 7
C. 6
D. 5

二叉树

- 已知一棵完全二叉树的第6层（设根为第一层）有8个叶子节点，则该完全二叉树的节点个数最多为（）。
  - A. 39
  - B. 52
  - C. 111
  - D. 119

二叉树

- 若一棵完全二叉树有768个节点，则该二叉树中叶子节点个数为（）。

A. 257
B. 258
C. 384
D. 385

二叉树

- 设一棵非空完全二叉树T的所有叶子节点都在同一层，且每个非叶子节点都有2个子节点。若T有k个叶节点，则T的节点总数为
  A. 2K-1
  B. 2K
  C. K^2
  D. 2^K - 1

二叉树

- 对于任意一棵高度为5且有10个节点的二叉树，若采用顺序存储结构保存，每个节点占1个存储单元（仅存放节点的数据信息），则存放该二叉树需要的存储单元数量至少是（）。

A. 31
B. 16
C. 15
D. 10

二叉树的遍历

- 若一棵二叉树的前序遍历序列为AEBDC，后序遍历序列为BCDEA，则根节点的孩子节点为（）。
  - A. 只有E
  - B. 有EB
  - C. 有EC
  - D. 无法确定

二叉树的遍历

- 要使一棵非空二叉树的前序序列和中序系列相同，其所有非叶子节点必须满足的条件是（）。
  - A. 只有左子树
  - B. 只有右子树
  - C. 节点度均为1
  - D. 节点度均为2

二叉树的遍历

- 前序序列为abcd的不同二叉树个数为（）。
  - A. 13
  - B. 14
  - C. 15
  - D. 16

二叉树的遍历

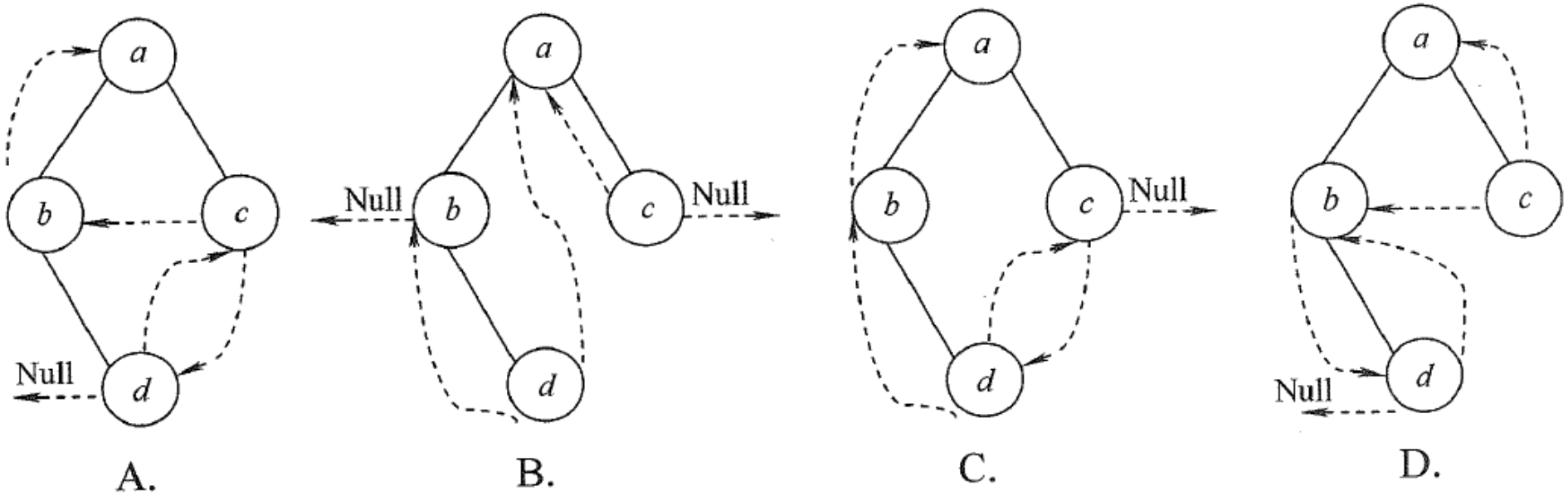
- 若 p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历序列形如“……p，v，q，……”，则下列叙述中，正确的是
  - A. p没有右孩子，q没有左孩子
  - B. p没有右孩子，q有左孩子
  - C. p有右孩子，q没有左孩子
  - D. p有右孩子，q有左孩子

线索二叉树

- 若X是后序线索二叉树中的叶节点，且X存在左兄弟节点Y，则X的右线索指向的是（）。
  - A. X的父节点
  - B. 以Y为根的子树的最左下节点
  - C. X的左兄弟节点Y
  - D. 以Y为根的子树的最右下节点

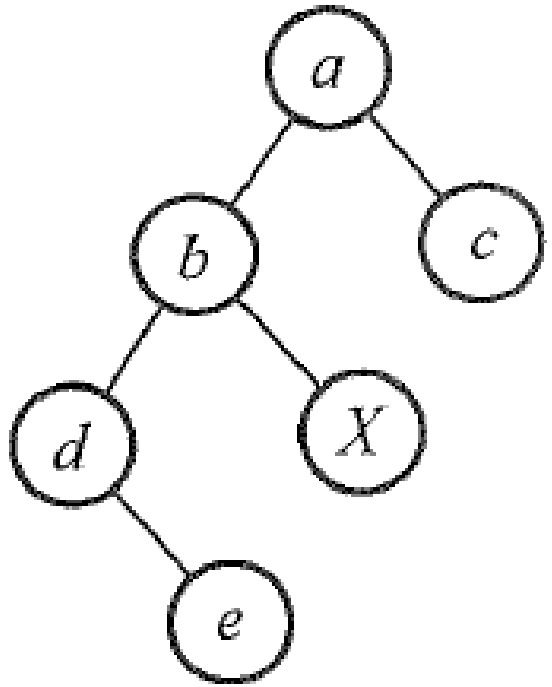
线索二叉树

- 下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（）。



线索二叉树

- 若对右图所示的二叉树进行中序线索化，则结点X的左右线索指向结点分别是（）。
  - A. e, c
  - B. e, a
  - C. d, c
  - D. b, a



森林

- 若森林F有15条边，25个结点，则F包含树的个数是（）
  - A. 8
  - B. 9
  - C. 10
  - D. 11

树和二叉树

- 已知一棵有2011个结点的树，其叶子结点个数为116，该树对应的二叉树中无右孩子的结点个数是（）。
  - A. 115
  - B. 116
  - C. 1895
  - D. 1896

树与二叉树

- 若将一棵树T转化为对应的二叉树BT，则下列对BT的遍历中，其遍历序列与T的后根遍历序列相同的是
  - A. 前序遍历
  - B. 中序遍历
  - C. 后序遍历
  - D. 按层遍历

森林和二叉树

- 将森林F转化为对应的二叉树T，F中叶子结点的个数等于（）
  - A. T中叶子结点的个数
  - B. T中度为1的结点个数
  - C. T中左孩子指针为空的结点个数
  - D. T中右孩子指针为空的结点个数

森林与二叉树

- 将森林转化为二叉树，若在二叉树中，节点u是节点v的父节点的父节点，则在原来的森林中，u和v可能具有的关系是（）。

  I. 父子关系

  II. 兄弟关系

  III. u的父节点与v的父节点是兄弟关系

  A. 只有II

  B. I和II

  C. I和III

  D. I、II和III

森林与二叉树

- 已知森林F及与之对应的二叉树T，若F的先根遍历序列是abcdef，后根遍历序列是badfec，则T的后序遍历序列是（）
  - A. badfec
  - B. bdfeca
  - C. bfedca
  - D. fedcba

并查集

- 并查集的结构是一种（）。
  - A. 二叉链表存储的二叉树
  - B. 双亲表示法存储的树
  - C. 顺序存储的二叉树
  - D. 孩子表示法存储的树

并查集

- 假设初始长度为10（0-9）的并查集，按1-2，3-4，5-6，7-8，8-9，1-8，0-5，1-9的顺序进行查找和合并，最终并查集共有（）个集合。

A. 1
B. 2
C. 3
D. 4

并查集

- 下列并查集的说法中，正确的是（）。
  - A. 并查集不能检测图中是否存在环路的问题
  - B. 通过路径优化后的并查集在最坏情况下的高度仍是O(n)
  - C. Find操作返回集合中元素个数的相反数，它用来作为某个集合的标志
  - D. 并查集基于树的双亲表示法

并查集

- 下列关于并查集的叙述中，（）是错误的。
  - 并查集是用双亲表示法存储的树
  - 并查集可以用于实现Kruskal算法
  - 并查集可以用于判断无向图的连通性
  - 在长度为n的并查集中进行查找操作的时间复杂度为O(logn)

哈夫曼树

- n（n>=2）个权值均不相同的字符构成哈夫曼树，关于该树的叙述中，错误的是（）。
  - A. 该树一定是一棵完全二叉树
  - B. 树中一定没有度为1的结点
  - C. 树中两个权值最小的结点一定是兄弟结点
  - D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

哈夫曼树

- 在有n个叶子结点的哈夫曼树中，非叶子结点的总数是（）。
  - A. n-1
  - B. n
  - C. 2n-1
  - D. 2n

哈夫曼树

- 对n个互不相同的符号进行哈夫曼编码。若生产的哈夫曼树共有115个结点，则n的值是（）。
  - A. 56
  - B. 57
  - C. 58
  - D. 60

哈夫曼树

- 5个字符有如下4种编码方案，不是前缀编码的是（）。
  - A. 01, 0000, 0001, 001, 1
  - B. 011, 000, 001, 010, 1
  - C. 000, 001, 010, 011, 100
  - D. 0, 100, 110, 1110, 1100

哈夫曼树

- 下列选项给出的是从根分别到达两个叶子结点路径上的权值序列，属于同一棵哈夫曼树的是（）。

A. 24、10、5和24、10、7

B. 24、10、5和24、12、7

C. 24、10、10和24、14、11

D. 24、10、5和24、14、6

哈夫曼树

- 已知字符集｛a, b, c, d, e, f, g, h｝，若各字符的哈夫曼编码依次是0100，10，0000，0101，001，011，11，0001，则编码序列01000110010010111110101的译码结果是（）。

A. a c g a b f h
B. a d b a g b b
C. a f b e a g d
D. a f e e f g d

哈夫曼树

- 已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为6，3，8，2，10，4，则对应字符集中各字符的哈夫曼编码可能是（）。

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 0000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01 ,000

哈夫曼树（变型）

- 已知三叉树T中6个叶结点的权分别是 2，3，4，5，6，7，T的带权（外部）路径长度最小是（）。
  - A. 27
  - B. 46
  - C. 54
  - D. 56

哈夫曼树

- 若某二叉树有5个叶结点，其权值分别为10，12，16，21，30，则其最小的带权路径长度为（）。
  - A. 89
  - B. 200
  - C. 208
  - D. 289

哈夫曼树

- 对任意给定的含n（n>2）个字符的有限集S，用二叉树表示S的哈夫曼编码集和定长编码集，分别得到二叉树T1和T2。下列描述中，正确的是（）。
  - A. T1和T2的结点数相同
  - B. T1的高度大于T2的高度
  - C. 出现频次不同的字符在T1中处于不同的层
  - D. 出现频次不同的字符在T2中处于不同的层

哈夫曼树

- 在由6个字符组成的字符集S中，各字符出现的频次分别为3，4，5，6，8，10，为S构造的哈夫曼编码的加权平均长度为（）
  - A. 2.4
  - B. 2.5
  - C. 2.67
  - D. 2.75

图

- 无向图G有16条边，其中度为4的顶点个数为3，度为3的顶点个数是4，其他顶点的度均小于3。图G所含顶点个数至少是（）
  - A. 10
  - B. 11
  - C. 13
  - D. 15

图

- 若G是一个有36条边的非连通简单无向图，则图G的顶点数至少是（）。
  - A. 11
  - B. 10
  - C. 9
  - D. 8

图

- 若无向图G=（V, E）中含有7个顶点，要保证图G在任何情况下都是连通的，则需要的边数最少是（）。
  - A. 6
  - B. 15
  - C. 16
  - D. 21

图

- 下列关于无向连通图的叙述中，正确的是（）。

I. 所有顶点的度之和为偶数

II. 边数大于顶点个数减1

III. 至少有一个顶点的度为1

A. 仅I　　B. 仅II　　C. 仅I和II　　D. 仅I和III

图

- 对于无向图G=(V,E)，下列选项中，正确的是（）。
  - A. 当|V|>|E|时，G一定是连通的
  - B. 当|V|<|E|时，G一定是连通的
  - C. 当|V|=|E|-1时，G一定是不连通的
  - D. 当|V|>|E|-1时，G一定是不连通的

图的遍历

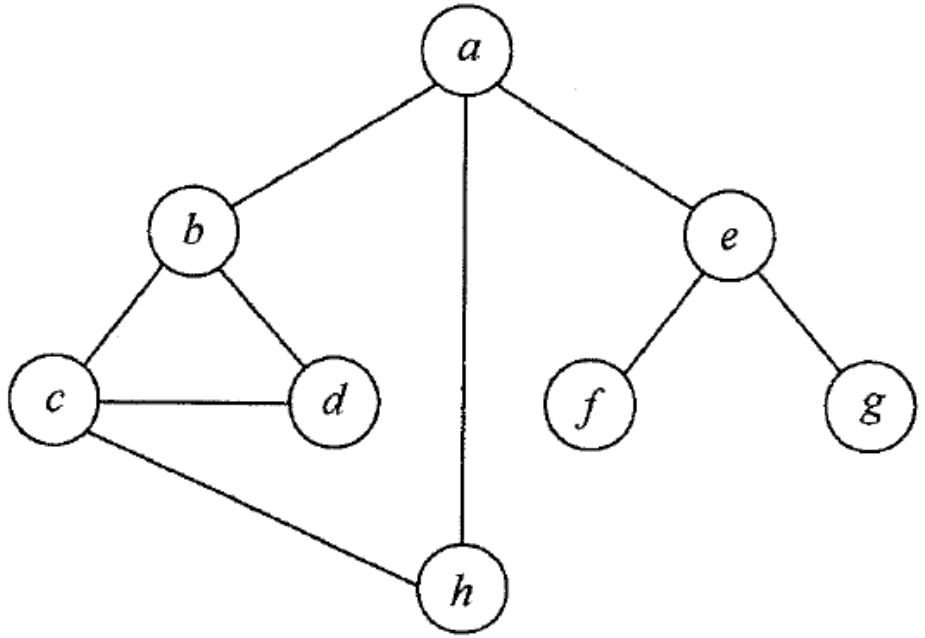
- 若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是（）。

A. h, c, a, b, d, e, g, f

B. e, a, f, g, b, h, c, d

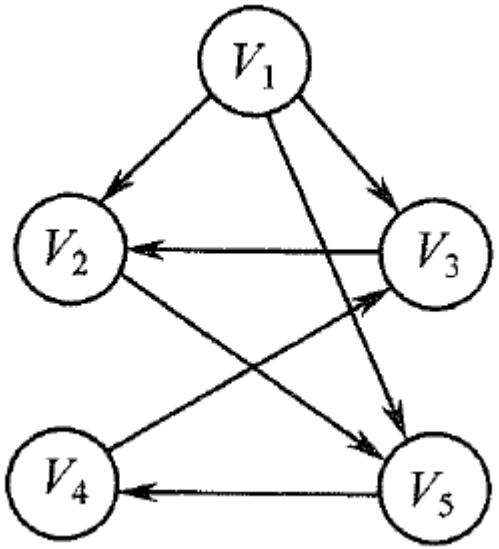
C. d, b, c, a, h, e, f, g

D. a, b, c, d, h, e, f, g



图的遍历

- 下列选项中，不是下图深度优先遍历序列的是（）。
  - A. V1, V5, V4, V3, V2
  - B. V1, V3, V2, V5, V4
  - C. V1, V2, V5, V4, V3
  - D. V1, V2, V3, V4, V5



图的遍历

- 设有向图G=(V, E)，顶点集V={v0, v1, v2, v3}，边集E={<v0,v1>, <v0,v2>,<v0,v3>,<v1,v3>}。若从顶点v0开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（ ）。

  A. 2
  B. 3
  C. 4
  D. 5

最小生成树

- 下列关于最小生成树的叙述中，正确的是（）。

I. 最小生成树的代价唯一

II. 所有权值最小的边一定会出现在所有的最小生成树中

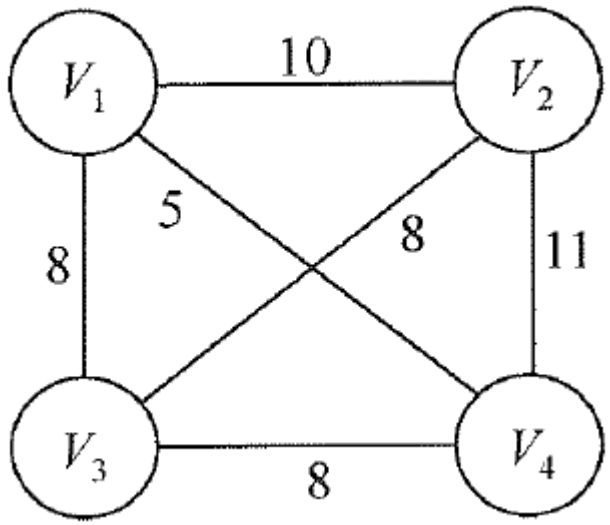
III. 使用Prim算法从不同顶点开始的最小生成树一定相同

IV. 使用Prime和Kruskal算法得到的最小生成树总不相同

A. 仅I　B. 仅II　C. 仅I, III　D. 仅II, IV

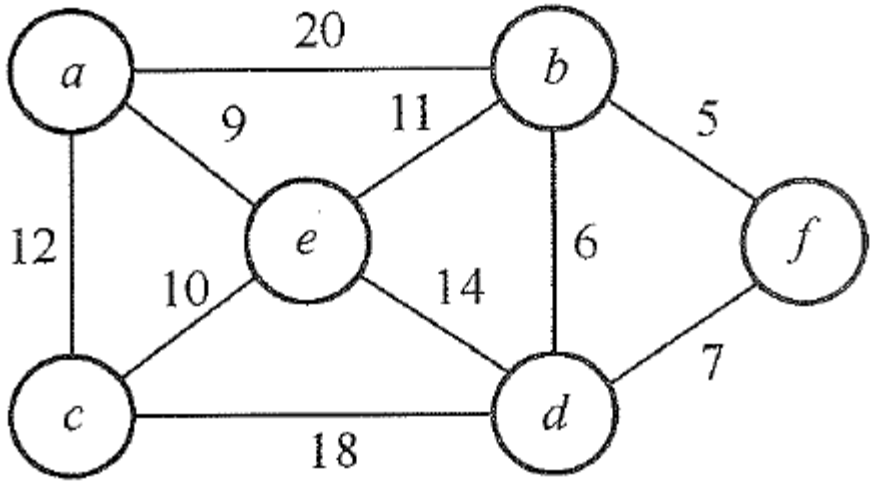
最小生成树

- 求右图的最小生成树时，可能是Kruskal算法第2次选中但不是Prim算法（从V4开始）第2次选中的边是（）。
  - A. (V1, V3)
  - B. (V1, V4)
  - C. (V2, V3)
  - D. (V3, V4)



最小生成树

- 已知无向图G如右所示，使用Kruskal算法求图G的最小生成树，加到最小生成树中的边依次是（）。
  - A. bf, bd, ae, ce, be
  - B. bf, bd, be, ae, ce
  - C. ae, be, ce, bd, bf
  - D. ae, ce, be, bf, bd



最短路径

- 下列关于广度优先算法的说法中，正确的是（） 。

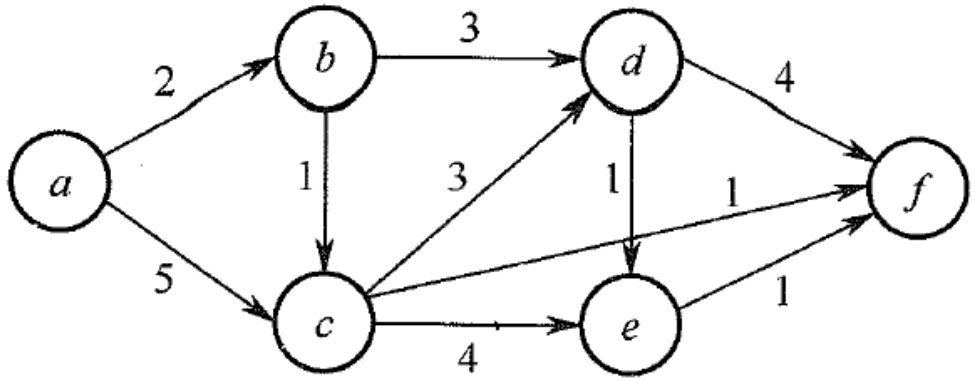
I. 当各边的权值相等时，广度优先算法可以解决单源最短路径问题
II. 当各边的权值不相等时，广度优先算法可以解决单源最短路径问题
III. 广度优先遍历算法类似于树中的后序遍历算法
IV. 实现图的广度优先算法时，使用的数据结构是队列

A. I, IV　　B. II, III, IV　　C. II, IV　　D. I, III, IV

最短路径

- 对下图所示的有向带权图，若采用Dijkstra算法从源点a到其他各顶点的最短路径，则得到的第一条最短路径的目标顶点是b，第二条最短路径的目标顶点是c，后序得到的其余各最短路径的目标顶点依次是（）。

  A. d, e, f
  B. e, d, f
  C. f, d, e
  D. f, e, d



最短路径

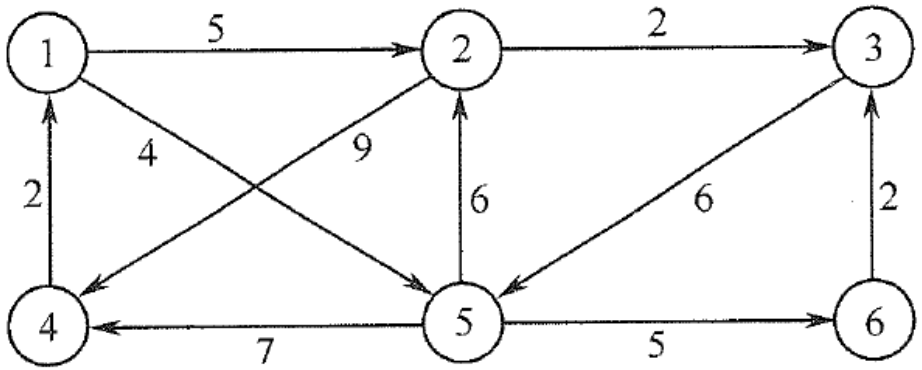
- 使用Dijkstra算法求下图中从顶点1到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（）。

A. 5，2，3，4，6

B. 5，2，3，6，4

C. 5，2，4，3，6

D. 5，2，6，3，4



最短路径

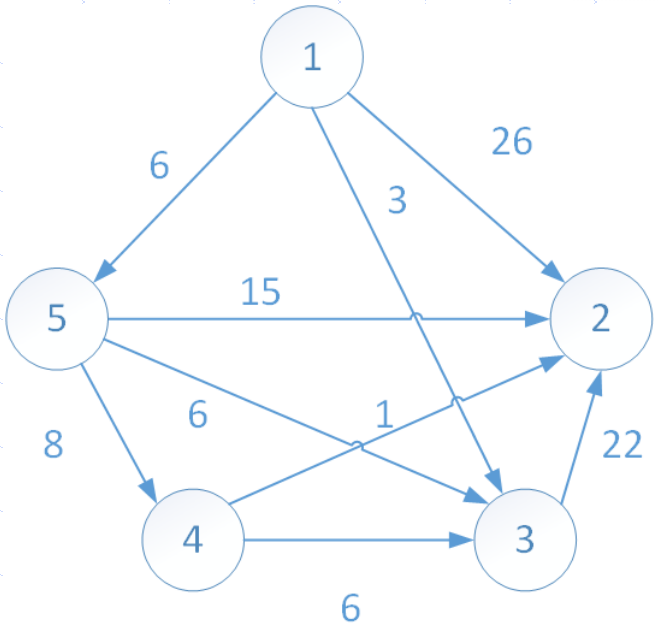
- 使用Dijkstra算法求下图中从顶点1到其余各顶点的最短路径，将当前找到的1到2，3，4，5的最短路径长度保存在数组dist中，求出第二条最短路径后，dist中的内容更新为（）。

A. 26，3，14，6

B. 25，3，14，6

C. 21，3，14，6

D. 15，3，14，6



最短路径

- Prim算法中，边上权重可以为负数吗？如果可以，请说明理由，如果不可以，请举例说明。

- Floyd算法中，边上权重可以为负数吗？如果可以，请说明理由，如果不可以，请举例说明。

有向无环图

- 用有向无环图描述表达式(x+y)((x+y)/x)，需要的顶点个数至少是（）。
  - A. 5
  - B. 6
  - C. 8
  - D. 9

有向无环图

- 修改递归方式实现的图的DFD算法，将输出（访问）顶点信息的语句移到退出递归前（即执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图G，若输出结果中包含的全部顶点，则输出的顶点序列是G的（）。

A. 拓扑有序序列
B. 逆拓扑有序序列
C. 广度优先搜索序列
D. 深度优先搜索序列

拓扑排序

- 下列关于图的叙述中，正确的是（） 。

I. 回路是简单路径

II. 存储稀疏图，用邻接矩阵比邻接表更省空间

III. 若有向图中存在拓扑序列，则该图不存在回路

A. 仅II　　B. 仅I、II　　C. 仅III　　D. 仅I、III

拓扑排序

- 若对n个顶点、e条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是（）。
  A. O(n)
  B. O(n+e)
  C. $O(n^2)$
  D. O(ne)

拓扑排序

- 以下关于拓扑排序的说法中，错误的是（）
  - I. 若某有向图存在环路，则该有向图一定不存在拓扑排序
  - II. 在拓扑排序算法中为暂存入度为零的顶点，可以使用栈，也可以使用队列
  - III. 若有向图的拓扑排序序列唯一，则图中每个顶点的入度和出度最多为1

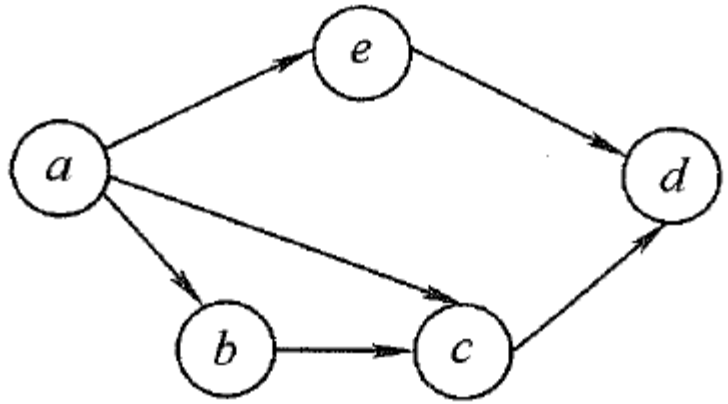
A. I, III　B. II, III　C. II　D. III

拓扑排序

- 若用邻接矩阵存储有向图，矩阵主对角线以下的元素全为0，则关于该图拓扑排序的结论是（）。
  - A. 存在，且唯一
  - B. 存在，且不唯一
  - C. 存在，可能不唯一
  - D. 无法确定是否存在

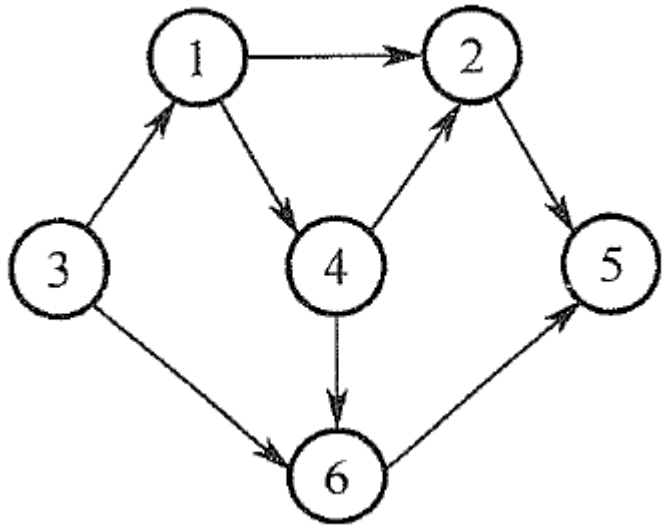
拓扑排序

- 对下图进行拓扑排序，可得到不同拓扑排序的个数是（ ）。
  - A. 4
  - B. 3
  - C. 2
  - D. 1



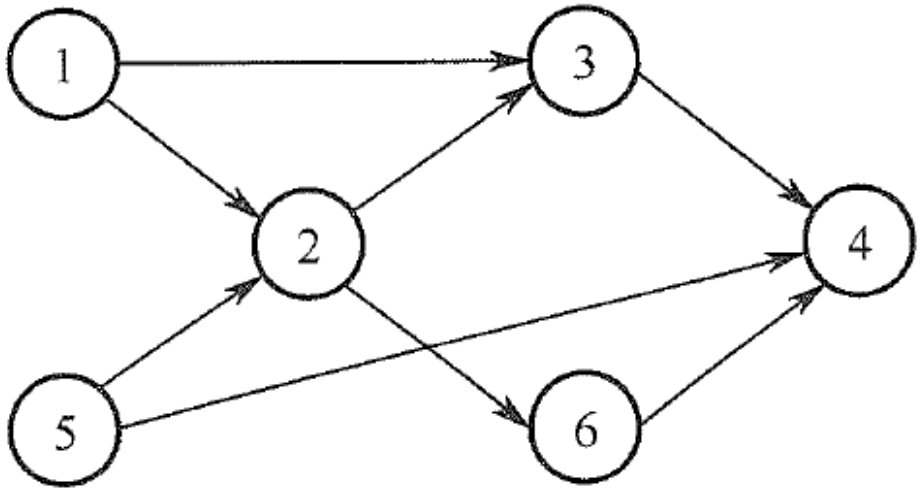
拓扑排序

- 对下图所示的有向图进行拓扑排序，得到的拓扑序列可能是（）。
  - A. 3，1，2，4，5，6
  - B. 3，1，2，4，6，5
  - C. 3，1，4，2，5，6
  - D. 3，1，4，2，6，5



拓扑排序

- 下列选项中，不是右图拓扑排序序列的是（）。
  - A. 1，5，2，3，6，4
  - B. 5，1，2，6，3，4
  - C. 5，1，2，3，6，4
  - D. 5，2，1，6，3，4

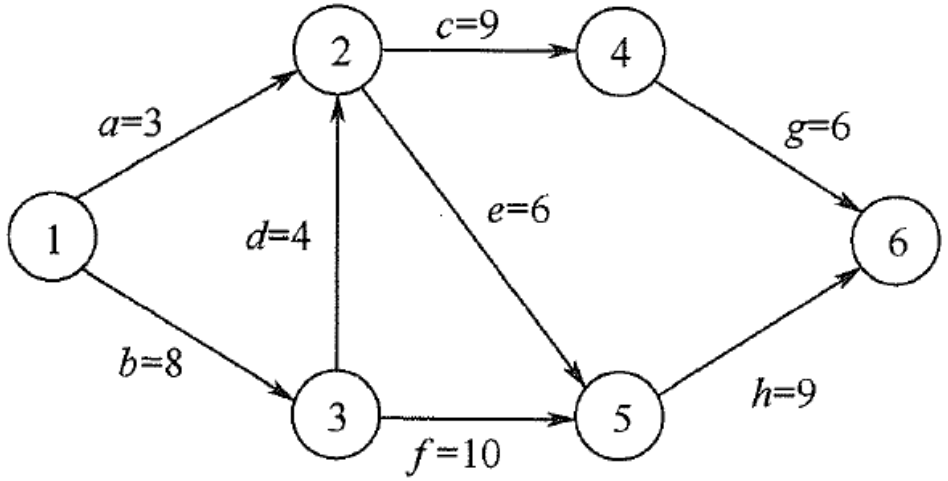


关键路径

- 若使用AOE网估算工程进度，则下列叙述中正确的是（）。
  - A. 关键路径是从源点到汇点边数最多的一条路径
  - B. 关键路径是从源点到汇点路径长度最长的路径
  - C. 增加任一关键活动的时间不会延长工程的工期
  - D. 缩减任一关键活动的时间将会缩短工程的工期

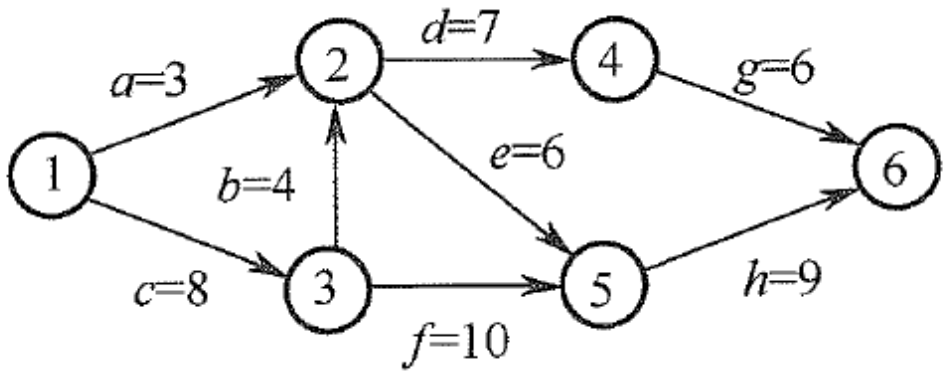
关键路径

- 下列AOE网表示一项包含8个活动的工程。通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（）。
  - A. c和e
  - B. d和c
  - C. f和d
  - D. f和h



关键路径

- 右图所示AOE网表示一项包括8个活动的工程。活动d最早开始和最迟开始时间分别是（）
  - A. 3和7
  - B. 12和12
  - C. 12和14
  - D. 15和15



不等概率查找

- 设包含4个数据元素的集合S={ "do"，"for"，" repeat"，"while"}，各元素的查找概率依次为：p1=0.35，p2 = 0.15，p3=0. 15，p4=0.35。将S保存在一个长度为4的顺序表中，采用折半查找法，查找成功时的平均查找长度为2.2。请回答：

  （1）若采用顺序存储结构保存S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

  （2）若采用链式存储结构保存S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

折半查找

- 已知一个长度为16的顺序表L，其元素按关键字有序排列，若采用折半查找法查找一个L中不存在的元素，则关键字的比较次数最多是（）。
  - A. 4
  - B. 5
  - C. 6
  - D. 7

折半查找

- 下列选项中，不能构成折半查找中关键字比较序列的是（）。
  - A. 500，200，415，180
  - B. 500，450，200，180
  - C. 180，500，200，450
  - D. 180，200，500，450

折半查找

- 在有n（n>1000）个元素的升序数组A中查找关键字x，查找算法的伪代码如右。本算法与折半查找相比，有可能具有更少比较次数的情形是（）。
  - A. 当x不在数组中
  - B. 当x接近数组开头处
  - C. 当x接近数组结尾处
  - D. 当x接近数组中间位置

```
k=0;
while(k<n 且 A[k]<x) k=k+3;
if(k<n 且 A[k]==x) 查找成功;
else if(k-1<n 且 A[k-1]==x) 查找成功;
     else if(k-2<n 且 A[k-2]==x) 查找成功;
          else 查找失败;
```

折半查找

- 对含600个元素的有序顺序表进行折半查找，关键字间的比较次数最多是（）
  - A. 9
  - B. 10
  - C. 30
  - D. 300

折半查找

- 下列数据结构中，不适合直接使用折半查找的是（）。
  I. 有序链表　　II. 无序数组　　III. 有序静态链表
  IV. 无序静态链表

  A.仅I, III
  B. 仅II, IV
  C. 仅II, III, IV
  D. I, II, III, IV

BST树

- 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的是（）。
  - A. 95、22、91、24、94、71
  - B. 92、20、91、34、88、35
  - C. 21、89、77、29、36、38
  - D. 12、25、71、68、33、34

BST树

- 在任意一棵非空二叉排序树T1中，删除某结点v后形成二叉排序树T2，再将v插入T2形成二叉排序树T3。下列关于T1与T3的叙述中，正确的是（）。

  I. 若v是T1的叶结点，则T1和T3不同。

  II. 若v是T1的叶结点，则T1和T3相同。

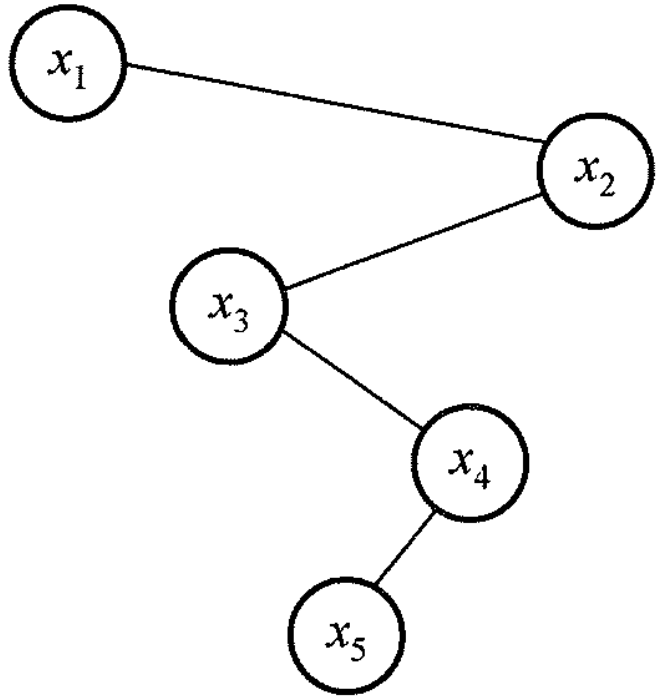
  III. 若v不是T1的叶结点，则T1和T3不同。

  IV. 若v不是T1的叶结点，则T1和T3相同。

  A. 仅I、III　B. 仅I、IV　C.仅II、III　D. 仅II、IV

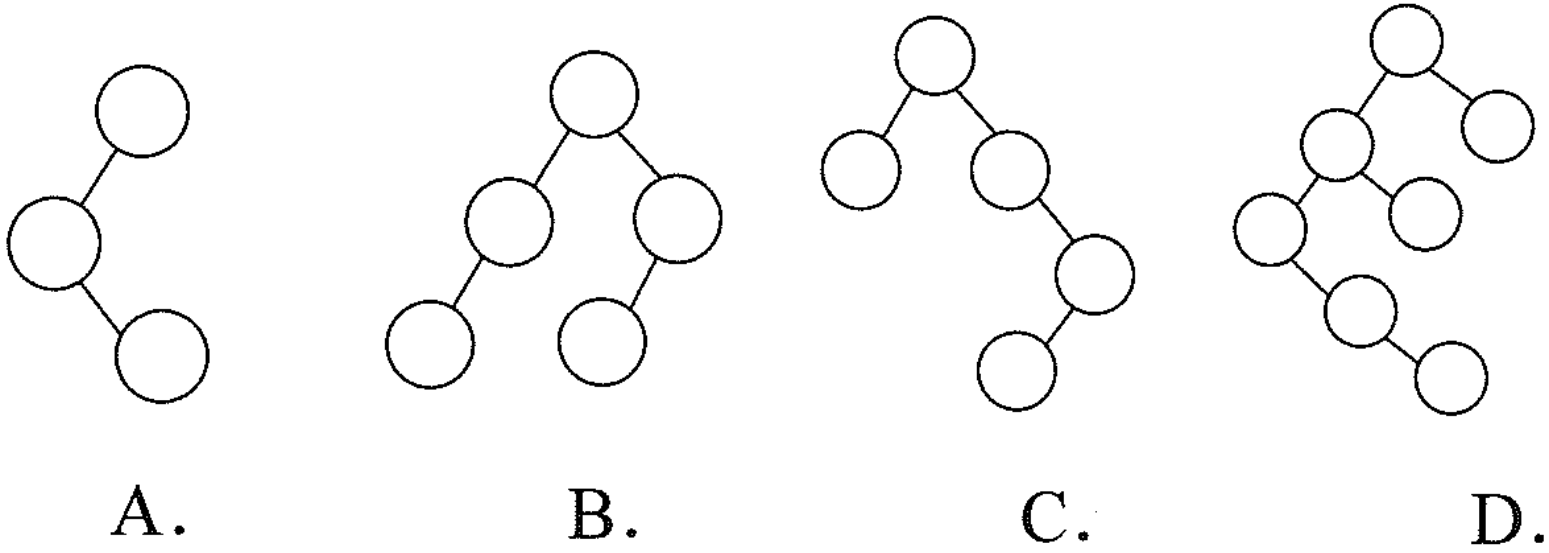
BST树

- 已知二叉排序树如图所示，元素之间应满足的大小关系是（）。
  - A. x1<x2<x5
  - B. x1<x4<x5
  - C. x3<x5<x4
  - D. x4<x3<x5



AVL树

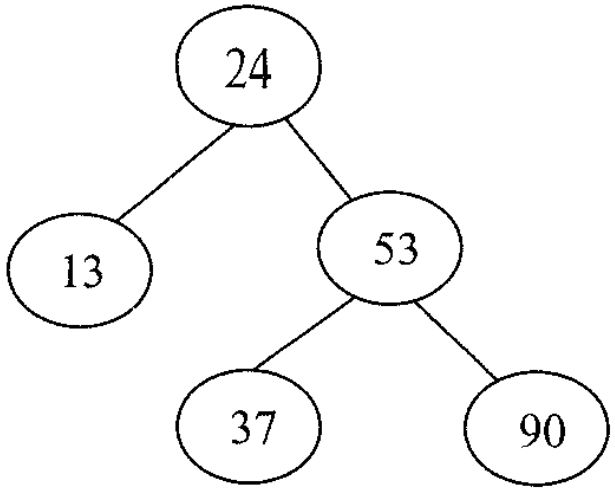
- 下列二叉树中，满足平衡二叉树定义的是（）。



A. B. C. D.

AVL树

- 如图所示AVL树中插入关键字48后得到一棵新的AVL树，则新树中关键字37的左右子结点分别是
  - A. 13，48
  - B. 24，48
  - C. 24，53
  - D. 24，90



AVL树

- 若平衡二叉树的高度为6，且所有非叶子结点的平衡因子为1，则该平衡二叉树的结点总数为（）。
  - A. 12
  - B. 20
  - C. 32
  - D. 33

AVL树

- 若将关键字1-7依次插入初始为空的平衡二叉树T，则T中平衡因子为0的分支结点的个数是（）。
  - A. 0
  - B. 1
  - C. 2
  - D. 3

AVL树

- 现有一棵无重复关键字的平衡二叉树，对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是（）。

A. 根结点的度一定为2
B. 树中最小元素一定是叶结点
C. 最后插入的元素一定是叶结点
D. 树中最大元素一定是无左子树

AVL树

- 在任意一棵非空平衡二叉树T1中，删除某结点v之后形成的平衡二叉树T2，再将v插入T2形成平衡二叉树T3。下列关于T1和T3的叙述中，正确的是（）。

  I. 若v是T1的叶结点，则T1与T3可能不相同

  II. 若v不是T1的叶结点，则T1与T3一定不相同

  III. 若v不是T1的叶结点，则T1与T3一定相同

  A. 仅I　　B. 仅II　　C. 仅I、II　　D. 仅I、III

红黑树

- 下列关于红黑树和AVL树的说法中，不正确的是（）。

  I. 一棵含有n个节点的红黑树的高度至多为$2\log_2(n+1)$

  II. 若一个节点为红色，则它的父节点和子节点都是黑色

  III. 红黑树的查询效率一般要优于含有相同结点数的AVL树

  IV. 若AVL树的某节点的左右孩子的平衡因子都是0，则该节点的平衡因子也是0

  A. I,III　　B. III　　C. II,IV　　D. III,IV

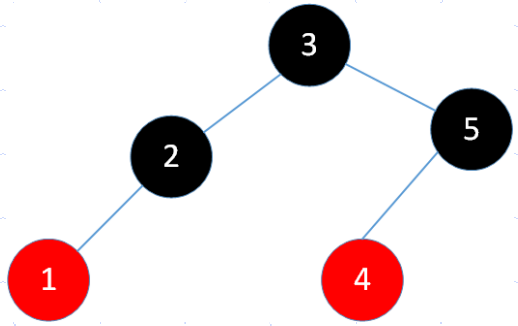
红黑树

- 下列关于红黑树的说法中，正确的是（）。
  - A. 红黑树的红节点的数目最多与黑节点的数目相同
  - B. 若红黑树的所有节点都是黑色的，则它一定是一棵满二叉树
  - C. 红黑树的任何一个分支节点都有两个非空孩子节点
  - D. 红黑树的子树也一定是红黑树

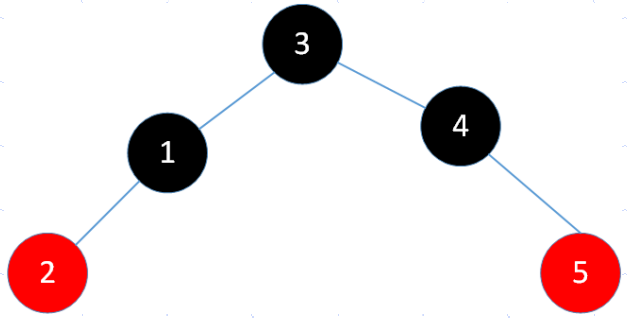
红黑树

- 下列4个选项中，满足红黑树定义的是（）。

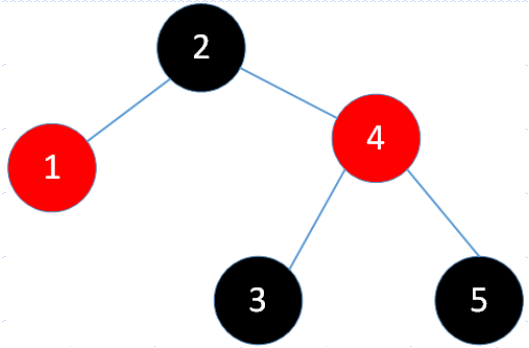
A



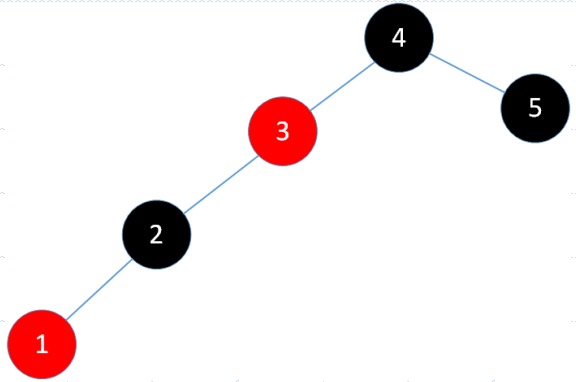
B



C



D

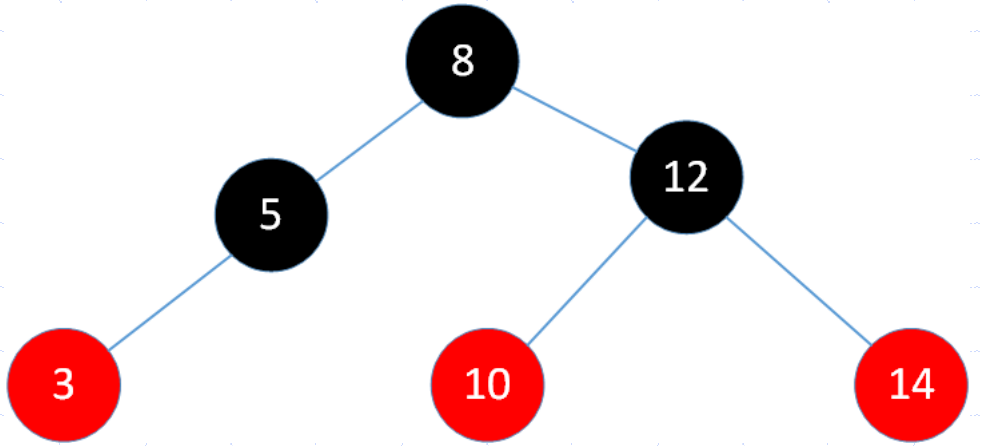


红黑树

- 将关键字1、2、3、4、5、6、7依次插入初始为空的红黑树T，则T中红节点个数是（）。
  - A. 1
  - B. 2
  - C. 3
  - D. 4

红黑树

- 在如图所示的红黑树中插入节点2且染成红色后，则下一步应进行的操作是（）
  - A. 左旋
  - B. 右旋
  - C. 变色
  - D. 无需调整

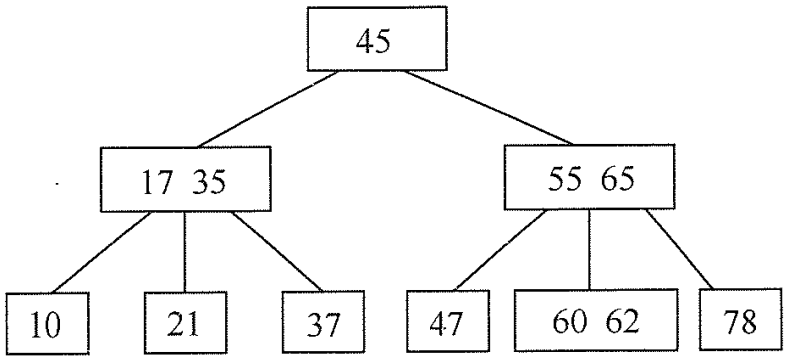


B树

- 下列叙述中，不符合m阶B树定义要求的是（）。
  - A. 根结点至多有m棵子树
  - B. 所有叶结点都在同一层上
  - C. 各结点内关键字均升序或降序排列
  - D. 叶结点之间通过指针链接

B树

- 已知三阶B树如图，删除78后得到新B树，其最右叶结点中的关键字是（）。
  - A. 60
  - B. 60，62
  - C. 62，65
  - D. 65



B树

- 在一棵高度为2的5阶B树中，所含关键字的个数至少是（）。
  - A. 5
  - B. 7
  - C. 8
  - D. 14

B树

- 在一棵具有15个关键字的4阶B树中，含关键字的结点个数最多是（）。
  - A. 5
  - B. 6
  - C. 10
  - D. 15

B树

- 高度为5的3阶B树含有的关键字个数至少是（）。
  - A. 15
  - B. 31
  - C. 62
  - D. 242

B树

- 依次将关键字5、6、9、13、8、2、12、15插入初始为空的4阶B树后，根结点中包含的关键字是（）。

A. 8

B. 6，9

C. 8，13

D. 9，12

B树

- B+树不同于B树的特点之一是（）。
  - A. 能支持顺序查找
  - B. 结点中含有关键字
  - C. 根结点至少有两个分支
  - D. 所有叶结点都在同一层上

B树

- 下列应用中，适合使用B+树的是（）。
  - A. 编译器中的词法分析
  - B. 关系数据库系统中的索引
  - C. 网络中的路由表快速查找
  - D. 操作系统的磁盘空闲块管理

B树

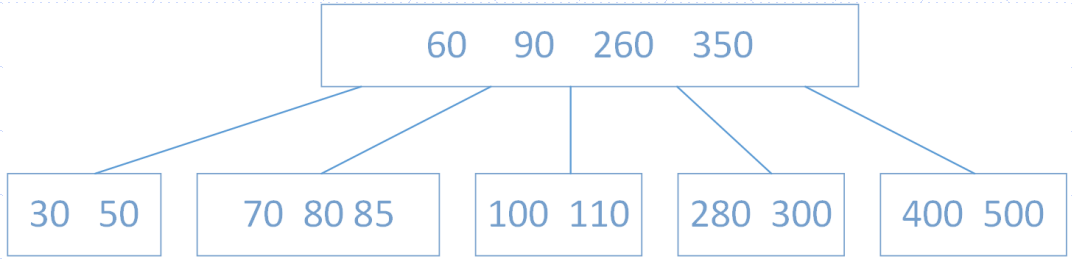
- 在如图所示的5阶B树T中，删除关键字260之后需要进行必要的调整，得到新树T1.下列选项中，不可能是T1根节点中关键字序列对是（）。

A. 60，90，280

B. 60，90，350

C. 60，85，110，350

D. 60，90，110，350



Hash

- 为提高散列表的查找效率，可以采取的正确措施是（）。

  I. 增大装载因子

  II. 设计冲突少的散列函数

  III. 处理冲突时避免产生聚集现象

  A. 仅I　B. 仅II　C. 仅I, II　D. 仅II, III

Hash

- 用哈希方法处理冲突时可能出现聚集现象，下列选项中，会受聚集现象直接影响的是（）。
  - A. 存储效率
  - B. 散列函数
  - C. 装载因子
  - D. 平均查找长度

Hash

- 现有长度为7，初始为空的散列表HT，散列函数H(key) = key%7，采用线性探测法解决冲突，将关键字22、43、15依次插入HT后，查找成功的平均查找长度为（）。
  - A. 1.5
  - B. 1.6
  - C. 2
  - D. 3

Hash

- 现有长度为11，初始为空的散列表HT，散列函数H(key) = key%7，采用线性探测法解决冲突，将关键字序列87、40、30、6、11、22、98、20依次插入HT后，查找失败的平均查找长度为（）。

A. 4
B. 5.25
C. 6
D. 6.29

Hash

- 现有长度为5、初始为空的散列表HT，散列函数H(k) = (k+4) %5，用线性探查再散列法解决冲突。若将关键字2022,12,25依次插入HT，然后删除关键字25，则HT中查找失败的平均查找长度是（）。

  A. 1
  B. 1.6
  C. 1.8
  D. 2.2

Hash

- 将关键字数列 20, 3, 11, 18, 9, 14, 7 依次存储到初始值为空、长度为11的散列表HT中，散列函数H(key)=(key*3)%11，H(key)计算出的初始散列地址为H_0，发生冲突时探查地址序列是H_1, H_2, H_3，其中$H_k=(H_0+k^2)\%11$，k=1,2,3… 。请回答下列问题。
  - 画出所构造的HT，并计算HT的填装因子。
  - 给出在HT中查找关键字14的关键字比较序列。
  - 在HT中查找关键字8，确认查找失败时散列地址是多少？

插入排序

- 对同一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（）。
  - A. 排序的总趟数
  - B. 元素的移动次数
  - C. 使用辅助空间的数量
  - D. 元素之间的比较次数

插入排序

- 用直接插入排序方法对下面四个序列进行排序(由小到大)，元素比较次数最少的是

A. 98、27、36、83、74、52、15、61

B. 27、36、15、52、61、98、83、74

C. 15、27、52、36、74、61、83、98

D. 83、61、74、52、15、27、98、36

希尔排序

- 希尔排序的组内排序用的是（）。
  - A. 直接插入排序
  - B. 折半插入排序
  - C. 快速排序
  - D. 归并排序

希尔排序

- 用希尔排序方法对一个序列进行排序时，若第一趟排序结果是9，1，4，13，7，8，20，23，15，则该趟排序采用的增量（间隔）可能是（）。
  - A. 2
  - B. 3
  - C. 4
  - D. 5

希尔排序

- 对初始序列（8、3、9、11、2、1、4、7、5、10、6）进行希尔排序。若第一趟排序结果为（1、3、7、5、2、6、4、9、11、10、8），第二趟排序结果为（1、2、6、4、3、7、5、8、11、10、9），则两趟排序采用的增量（间隔）分别为

A. 3，1

B. 3，2

C. 5，2

D. 5，3

快速排序

- 为实现快速排序算法，待排序序列宜采用的存储方式是（）。
  - A. 顺序存储
  - B. 散列存储
  - C. 链式存储
  - D. 索引存储

快速排序

- 采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是（）。

A. 递归次数与初始数据的排列次数无关

B. 每次划分后，先处理较长的分区可以减少递归次数

C. 每次划分后，先处理较短的分区可以减少递归次数

D. 递归次数与每次划分后得到的分区的处理顺序无关

快速排序

- 下列选项中，不可能是快速排序第2趟排序结果的是（）。

A. 2，3，5，4，6，7，9

B. 2，7，5，6，4，3，9

C. 3，2，5，4，7，6，9

D. 4，2，3，5，7，6，9

快速排序

- 下列选项中，不可能是快速排序第2趟排序结果的是（）。

A. 5，2，16，12，28，60，32，72

B. 2，16，5，28，12，60，32，72

C. 2，12，16，5，28，32，72，60

D. 5，2，12，28，16，32，72，60

快速排序

- 已知由n(n>=2)个正整数构成的集合$A = \{a_k | 0<=k<n\}$，将其划分为两个不相交的子集A_1和A_2，元素个数分别是n_1和n_2，A_1和A_2中的元素之和分别为S_1和S_2。试设计一个尽可能高效的划分算法，满足$|n_1-n_2|$最小且$|S_1-S_2|$最大，并说明所设计算法的时间复杂度和空间复杂度。

快速排序

- 使用快速排序算法对数据进行排序，若经过一次划分后得到的数据序列是68,11,70,23,80,77,48,81,93,88，则该次划分的枢轴是（）。
  - A. 11
  - B. 70
  - C. 80
  - D. 81

快速排序

- 使用快速排序对含 n(n≥3) 个元素的数组M进行排序，若第一趟排序将除M中枢轴外的n-1个元素划分为均不为空的P和Q两个块，则下列叙述中，正确的是（）。
  - A. P 和 Q 块间有序
  - B. P 和 Q 均块内有序
  - C. P 和 Q 的元素个数大致相等
  - D. P 和 Q 中均不存在相等的元素

选择排序

- 对序列27,19,8,11,32,1,5}进行排序,进行一趟后数据的排列变为{5,19,1,11,32,8,27};则采用的是()排序
  - A. 希尔排序
  - B. 选择排序
  - C. 冒泡排序
  - D. 快速排序

堆排序

- 下列关于大根堆（至少2个元素）的叙述中，正确的是（）。

I. 可以将堆视为一棵完全二叉树
II. 可以采用顺序存储方式保存堆
III. 可以将堆视为一棵二叉排序树
IV. 堆中的次大值一定在根的下一层

A. 仅I, II　B. 仅II, III　C. 仅I, II, IV　D. 仅I, III, IV

堆排序

- 已知关键字序列5、8、12、19、28、20、15、22是小根堆，插入关键字3，调整后得到的小根堆是（）。

A. 3、5、12、8、28、20、15、22、19

B. 3、5、12、19、20、15、22、8、28

C. 3、8、12、5、20、15、22、28、19

D. 3、12、5、8、28、20、15、22、19

堆排序

- 已知小根堆为8、15、10、21、34、16、12，删除关键字8后需重建堆，在此过程中，关键字之间的比较次数为（）。

A. 1
B. 2
C. 3
D. 4

堆排序

- 已知序列25，13，10，12，9是大根堆，在序列尾部插入新元素18，将其再调整为大根堆，调整过程中元素之间的比较次数是（）。
  - A. 1
  - B. 2
  - C. 4
  - D. 5

堆排序

- 在将数据序列（6，1，5，9，8，4，7）建成大根堆时，正确的序列变化过程是：

A. 6, 1, 7, 9, 8, 4, 5 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

B. 6, 9, 5, 1, 8, 4, 7 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 → 9, 6, 5, 1, 8, 4, 7 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 → 7, 1, 6, 9, 8, 4, 5 → 7, 9, 6, 1, 8, 4, 5 → 9, 7, 6, 1, 8, 4, 5 → 9, 8, 6, 1, 7, 4, 5

堆排序

- 将关键字6,9,1,5,8,4,7依次插入初始为空的大根堆H，得到的H是（）。
  - A. 9,8,7,6,5,4,1
  - B. 9,8,7,5,6,1,4
  - C. 9,8,7,5,6,4,1
  - D. 9,6,7,5,8,4,1

堆排序

- 已知关键字序列 28, 22, 20, 19, 8, 12, 15, 5是大根堆（最大堆），对该堆进行两次删除操作后，得到的新堆是（）。

A. 20, 19, 15, 12, 8, 5
B. 20, 19, 15, 5, 8, 12
C. 20, 19, 12, 15, 8, 5
D. 20, 19, 8, 12, 15, 5

归并排序

- 在内部排序时，如果选择了归并排序而并未选择插入排序，则可能的理由是（）。

  I. 归并排序的程序代码更短

  II. 归并排序的占用空间更少

  III. 归并排序的运行效率更高

  A. 仅II　B. 仅III　C. 仅I, II　D. 仅I, III

归并排序

- 使用二路归并排序对含n个元素的数组M进行排序时，二路归并程序的功能是（）。
  - A. 将两个有序表合并为一个新的有序表
  - B. 将M划分为两部分，两部分的元素个数大致相等
  - C. 将M划分为n部分，每个部分中仅含有一个元素
  - D. 将M划分为两部分，一部分元素的值均小于另一部分元素的值

归并排序

- 现有由关键字组成的3个有序序列 (3,5), (7,9) 和 (6)，若按从左至右的次序选择有序序列进行二路归并排序，则关键字之间的总比较次数是（）。
  - A. 3
  - B. 4
  - C. 5
  - D. 6

归并排序

- 一组经过第一趟2路归并排序后的记录的关键字是{25，50，15，35，80，85，20，40，36，70}，其中包含5个长度为2的有序表，则第二趟归并后的结果是（）。

A. 15,25,35,50,80,20,85,40,70,36
B. 15,25,35,50,20,40,80,85,36,70
C. 15,25,50,35,80,85,20,36,40,70
D. 15,25,35,50,80,20,36,40,70,85

基数排序

- 对于给定的关键字序列110，119，007，911，114，120，122进行基数排序，第2趟分配收集后得到的关键字序列是（）。
  - A. 007，110，119，114，911，120，122
  - B. 007，110，119，114，911，122，120
  - C. 007，110，911，114，119，120，122
  - D. 110，120，911，122，114，007，119

基数排序

- 设数组S[] = { 93,946,372,9,146,151,301,485,236,327,43,892 }，采用最低位优先LSD基数排序将S排列成升序序列。第一趟分配、收集后，元素372之前、之后紧邻的元素分别是（）。
  A. 43,892
  B. 236,301
  C. 301,892
  D. 485,301

排序综合

- 对10TB的数据文件进行排序，应使用的方法是（）。
  - A. 希尔排序
  - B. 堆排序
  - C. 快速排序
  - D. 归并排序

排序综合

- 若数据元素序列{11，12，13，7，8，9，23，4，5}是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是（）。
  - A. 冒泡排序
  - B. 插入排序
  - C. 选择排序
  - D. 2路归并排序

排序综合

- 对一组数据{2，12，16，88，5，10}进行排序，若前三趟结果如下：

  一：2，12，16，5，10，88

  二：2，12，5，10，16，88

  三：2，5，10，12，16，88

  则采用的排序方法可能是

  A. 冒泡排序
  B. 希尔排序
  C. 归并排序
  D. 基数排序

排序综合

- 在内部排序中，每趟排序结束都能至少确定一个元素最终位置的方法是（）。

I. 简单选择排序
II. 希尔排序
III. 快速排序
IV. 堆排序
V. 2路归并排序

A. 仅I, III, IV　B. 仅I, III, V　C. 仅II, III, IV　D. 仅III, IV, V

排序综合

- 下列排序算法中，元素的移动次数与关键字的初始排列顺序无关的是（）。
  - A. 直接插入排序
  - B. 冒泡排序
  - C. 基数排序
  - D. 快速排序

排序综合

- 下列排序方法中，若将顺序存储改为链式存储，则算法的时间效率会降低的是（）。

  I. 插入排序
  II. 选择排序
  III. 冒泡排序
  IV. 希尔排序
  V. 堆排序

  A. 仅I, II　B. 仅II, III　C. 仅III, IV　D. 仅IV, V

排序综合

- 选择一个排序算法时，除算法的时空效率外，下列因素中，还需要考虑的是（）。

  I. 数据的规模
  II. 数据的存储方式
  III. 算法的稳定性
  IV. 数据的初始状态

  A. 仅III　B. 仅I, II　C. 仅II, III, IV　D. I, II, III, IV

排序综合

- 对大部分元素已有序的数组排序时，直接插入排序比简单选择排序效率更高，其原因是（）。

  I. 直接插入排序过程中元素之间的比较次数更少

  II. 直接插入排序过程中所需的辅助空间更少

  III. 直接插入排序过程中元素的移动次数较少

  A. 仅I　B. 仅III　C. 仅I, II　D. I, II, III

排序综合

- 对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（）。

  I. 大部分元素已有序

  II. 待排序元素数量很少

  III. 要求空间复杂度为O(1)

  IV. 要求排序算法是稳定的

  A. 仅I, II　　B. 仅III, IV　　C.仅I, II ,IV　　D. I, II, III, IV

排序综合

- 下列排序算法中，不稳定的是（）。

I. 希尔排序
II. 归并排序
III. 快速排序
IV. 堆排序
V. 基数排序

A. 仅I, II　　B. 仅II, V　　C. 仅I, III, IV　　D. 仅III, IV, V

填充复习

真题演练

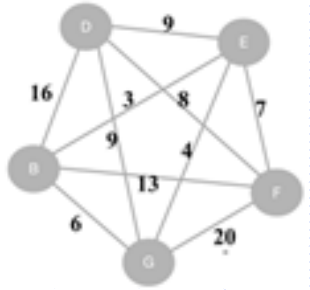
- 用数组q[0 … m-1]表示的顺序存储队列，队列中元素个数最多为m，用front表示队头指针，指向队头元素的前一位置，初值为-1，用rear表示队尾指针，指向队尾元素，判断队列发生假溢出的条件为 _____。
- 设一个栈的进栈序列为p1, p2, …, pn，其出栈序列为2, 3, …, n+1。若已知pn的值是2，那么pi的值是_____。
- 已知一棵完全二叉树的第6层（设根为第1层）有12个叶结点，则这棵树的结点至多有_____个。

真题演练

- 给定结点个数为n、高度为$h=\log_2(n+1)-1$的满二叉树，如果从0开始按中序遍历顺序对结点编号，则树根的左孩子编号用h表示为 _____。
- 如果二叉树中结点n的左右孩子都非空，则n的中序后继p的___孩子一定为空。
- 假设用于通信的电文由5个字符组成，已知其中一个字符的哈夫曼编码是1101，则该哈夫曼编码树的平均编码长度是 _____。

真题演练

- 把数字20, 22, 65, 55, 75, 34, 100, 104, 2022依次插入一个空的平衡二叉树(AVL tree)后（左结点小于根结点小于右结点），根结点的值为_____。
- 规定根结点高度为0，如果插入新结点后，一棵AVL树的高度为2，并且由于右子树为空导致不平衡，需要进行左右双旋，旋转后该AVL树的根结点的平衡因子值为_____。
- 在右图所示的带权连通图中，其最小生成树中各边权值之和是_____。



真题演练

- n个结点e条边的无向图用邻接表表示时，空间复杂度是O(__)。
- 在AOE网络中，若任意一个关键活动提前完成，则整个工程_____会提前完成；若任意一个关键活动延期完成，则整个工程_____会延期完成。（可填“一定”或者“不一定”）。
- 在n个数中用堆排序选择最小的k个数(k<n)，时间性能最好的算法时间复杂度可达到O(___)。
- 在冒泡排序、插入排序、选择排序、快速排序、归并排序、堆排序、以及基数排序中，不稳定的排序算法是_____（要求列出符合要求的两种算法名称）。

代码复习

代码复习

- 数组基本操作
- 链表基本操作
- 数制转换
- （二叉）树的遍历
- 哈夫曼树
- 图的数据结构表示和（广度）遍历
- 折半查找
- BST树
- AVL树*
- 插入/选择/冒泡排序
- 快速排序求分割
- 堆调整

数组的转置

42. （13 分）设将 n（n>1）个整数存放到一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p（0<p<n）个位置，即将 R 中的数据由（X0, X1…Xn-1）变换为（Xp, Xp+1 …Xn-1, X0, X1…Xp-1）。要求：

(1) 给出算法的基本设计思想。

(2) 根据设计思想，采用 C 或 C++或 JAVA 语言描述算法，关键之处给出注释。

(3) 说明你所设计算法的时间复杂度和空间复杂度。

数组的对齐

- 石头剪刀布是常见的猜拳游戏。石头胜剪刀，剪刀胜布，布胜石头。如果两个人出拳一样，则不分胜负。胜者得1分，负者减1分，平局则双方不得分。小A和小B正好在玩石头剪刀布。已知他们的出拳都是有周期性规律的，比如：“石头-布-石头-剪刀-石头-布-石头-剪刀……”，就是以“石头-布-石头-剪刀”为周期不断循环的。问小A和小B比了N轮之后，小A的得分？

输入

输入包含三行。第一行包含1个整数：N表示比了N轮。0 < N < 10^9。
第二行包含1个字符串，表示小A出拳的规律。（字符长度小于1000）
第三行包含1个字符串，表示小B出拳的规律。（字符长度小于1000）
其中，0表示“石头”，2表示“剪刀”，5表示“布”。

输出

输出一行，对于小A的得分。

标识数组

- 给定一个含n（n>=1)个整数的数组，设计一个在时间上尽可能高效的算法，找出数组中未出现的最小整数。如数组｛-5，3，2，3｝中未出现的最小是1，数组｛1，2，3｝中未出现的是4

暴力法

【2013 统考真题】已知一个整数序列 $A=(a_0, a_1, \cdots, a_{n-1})$，其中 $0 \le a_i < n$ $(0 \le i < n)$。若存在 $a_{p1}=a_{p2}= \cdots =a_{pm}=x$ 且 $m > n/2$ $(0 \le p_k < n, 1 \le k \le m)$，则称 x 为 A 的主元素。例如 $A=(0, 5, 5, 3, 5, 7, 5, 5)$，则 5 为主元素；又如 $A=(0, 5, 5, 3, 5, 1, 5, 7)$，则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，找出 A 的主元素。若存在主元素，则输出该元素；否则输出-1。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

单链表的运算

- 创建：尾插法、头插法
- 查找：按位查找、按值查找、求链长
- 插入：按位插入、指定节点后插、指定节点前插
- 删除：按位删除、指定节点删除

单链表的创建 — 尾插法

```c
typedef struct LNode{              //定义单链表结点类型
    ElemType data;                 //每个节点存放一个数据元素
    struct LNode *next;            //指针指向下一个节点
}LNode, *LinkList;

//初始化一个单链表（带头结点）
bool InitList(LinkList &L) {
    L = (LNode *) malloc(sizeof(LNode));    //分配一个头结点
    if (L==NULL)                   //内存不足，分配失败
        return false;
    L->next = NULL;                //头结点之后暂时还没有节点
    return true;
}

void test(){
    LinkList L;      //声明一个指向单链表的指针
    //初始化一个空表
    InitList(L);
    //......后续代码......
}
```



单链表的创建 — 尾插法

```
//在第 i 个位置插插入元素 e（带头结点）
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;     //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) {  //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)      //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;       //将结点s连到p之后
    return true;     //插入成功
}
```

每次都从头开始之后遍历，时间复杂度为 $O(n^2)$

尾插法建立单链表：

初始化单链表

设置变量 length 记录链表长度

While 循环{

每次取一个数据元素e;

ListInsert (L, length+1, e) 插到尾部;

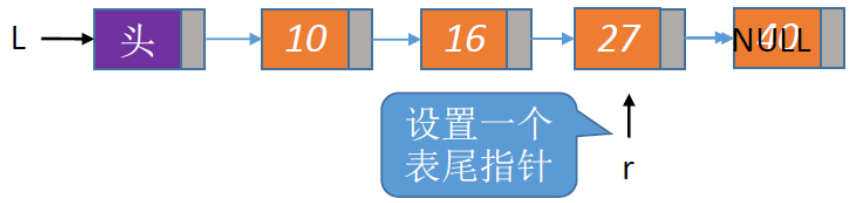
length++;

}

单链表的创建 — 尾插法

```
//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL)    //内存分配失败
        return false;
    s->data = e;          //用结点s保存数据元素e
    s->next=p->next;
    p->next=s;            //将结点s连到p之后
    return true;
}
```

后插操作



单链表的创建 — 尾插法

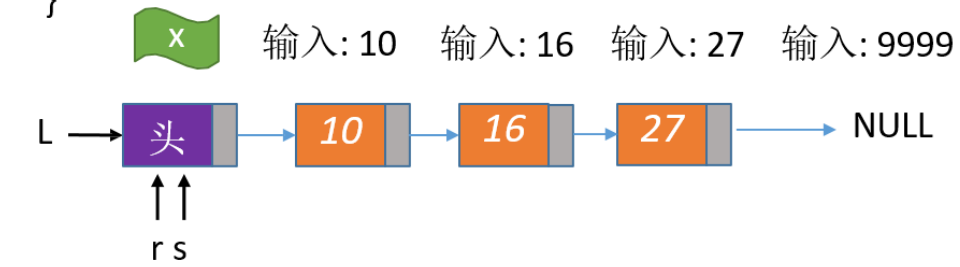
```
LinkList List_TailInsert(LinkList &L){      //正向建立单链表
    int x;                                  //设ElemType为整型
    L=(LinkList)malloc(sizeof(LNode));      //建立头结点
    LNode *s,*r=L;                          //r为表尾指针
    scanf("%d",&x);                         //输入结点的值
    while(x!=9999){                         //输入9999表示结束
        s=(LNode *)malloc(sizeof(LNode));
        s->data=x;
        r->next=s;
        r=s;                                //r指向新的表尾结点
        scanf("%d",&x);
    }
    r->next=NULL;                           //尾结点指针置空
    return L;
}
```

初始化空表

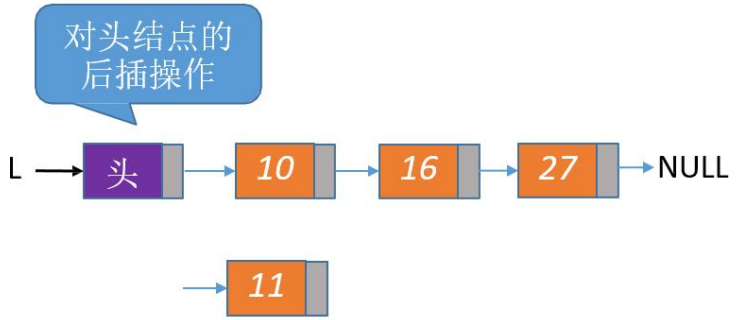
在r结点之后插入元素x

永远保持r 指向最后一个结点

时间复杂度：O(n)



单链表的创建 — 头插法




```
//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL)    //内存分配失败
        return false;
    s->data = e;          //用结点s保存数据元素e
    s->next=p->next;
    p->next=s;            //将结点s连到p之后
    return true;
}
```

头插法建立单链表：

初始化单链表

While 循环{
    每次取一个数据元素e;
    InsertNextNode (L, e);
}

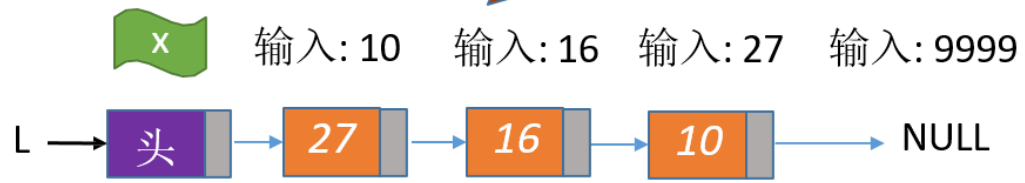
单链表的创建 — 头插法

```
LinkList List_HeadInsert(LinkList &L){    //逆向建立单链表
    LNode *s;
    int x;
    L=(LinkList)malloc(sizeof(LNode));    //创建头结点
    L->next=NULL;                         //初始为空链表
    scanf("%d",&x);                       //输入结点的值
    while(x!=9999){                       //输入9999表示结束
        s=(LNode*)malloc(sizeof(LNode)); //创建新结点
        s->data=x;
        s->next=L->next;
        L->next=s;                        //将新结点插入表中，L为头指针
        scanf("%d",&x);
    }
    return L;
}
```

如果去掉这一句呢？

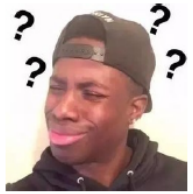
养成好习惯，只要是初始化单链表，都先把头指针指向NULL

重要应用！！！
链表的逆置



插入指定节点前

```
//前插操作：在p结点之前插入元素 e
bool InsertPriorNode (LNode *p, ElemType e)
```

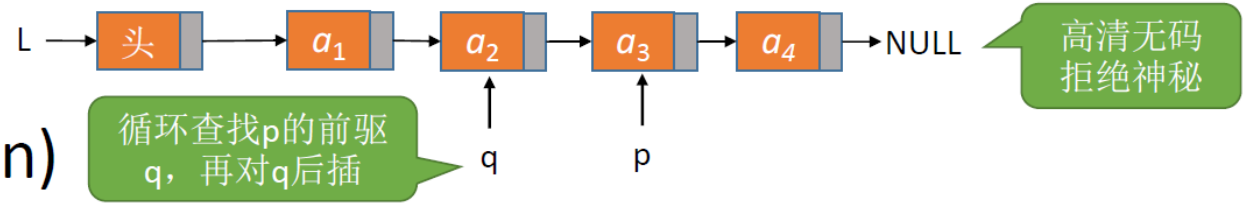







```
//前插操作：在p结点之前插入元素 e
bool InsertPriorNode (LinkList L, LNode *p, ElemType e)
```

传入头指针



高清无码拒绝神秘

时间复杂度：O(n)

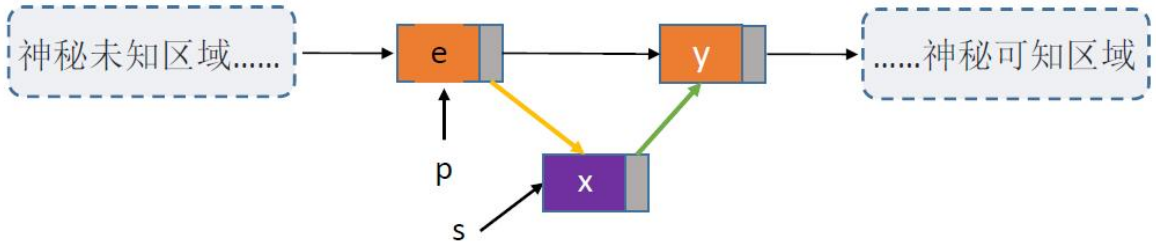
循环查找p的前驱q，再对q后插

插入指定节点前

```
//前插操作：在p结点之前插入元素 e
bool InsertPriorNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL)    //内存分配失败
        return false;
    s->next=p->next;
    p->next=s;              //新结点 s 连到 p 之后
    s->data=p->data;        //将p中元素复制到s中
    p->data=e;              //p 中元素覆盖为 e
    return true;
}
```

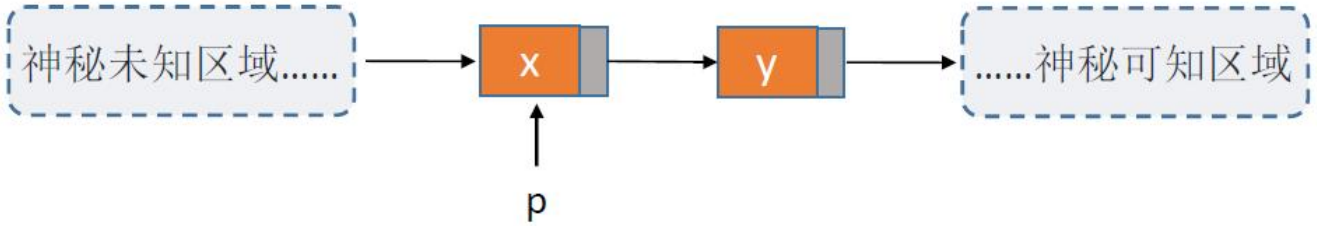


时间复杂度：O(1)



指定节点删除

```
//删除指定结点 p
bool DeleteNode (LNode *p)
```





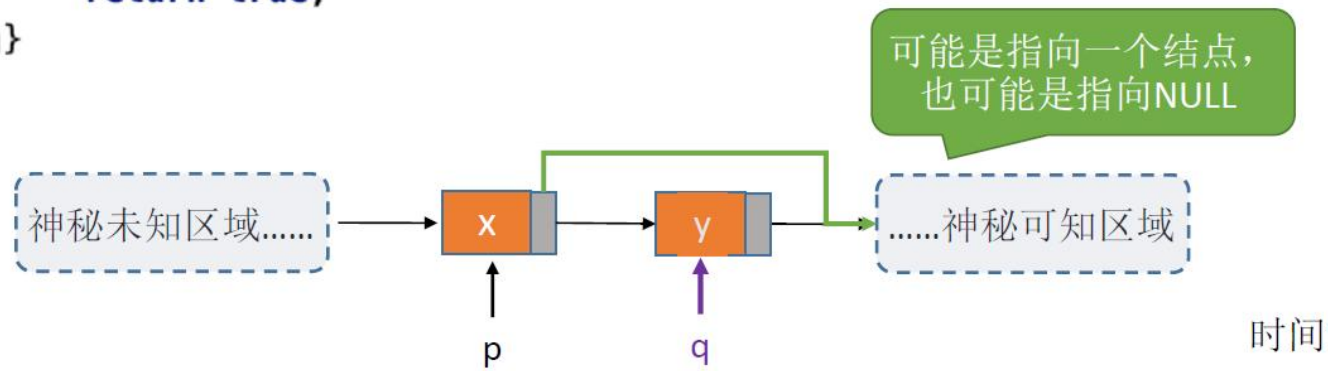
删除结点p，需要修改其前驱结点的 next 指针

方法1：传入头指针，循环寻找 p 的前驱结点
方法2：偷天换日（类似于结点前插的实现）

指定节点删除

```
//删除指定结点 p
bool DeleteNode (LNode *p){
    if (p==NULL)
        return false;
    LNode *q=p->next;          //令q指向*p的后继结点
    p->data=p->next->data;     //和后继结点交换数据域
    p->next=q->next;           //将*q结点从链中“断开”
    free(q);                   //释放后继结点的存储空间
    return true;
}
```

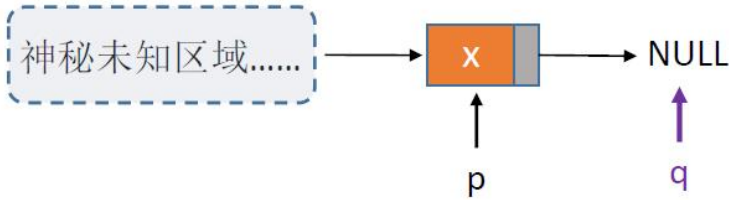




时间复杂度：O(1)

指定节点删除

```
//删除指定结点 p
bool DeleteNode (LNode *p){
    if (p==NULL)
        return false;
    LNode *q=p->next;          //令q指向*p的后继结点
    p->data=p->next->data;     //和后继结点交换数据域
    p->next=q->next;           //将*q结点从链中“断开”
    free(q);                   //释放后继结点的存储空间
    return true;
}
```



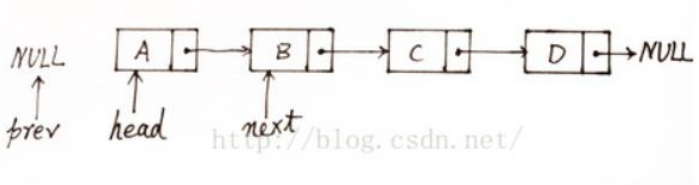




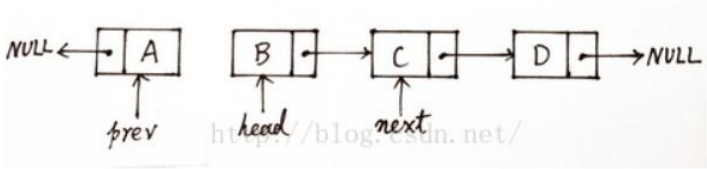
单链表逆序

- 时间复杂度：O(n)

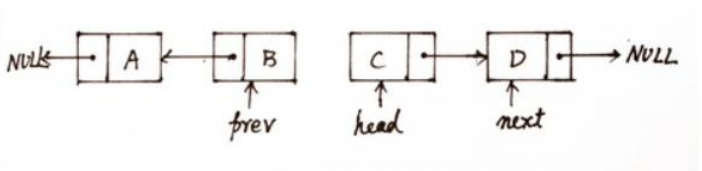
```
LinkNode* ReverseLink(LinkNode* head)
{
    LinkNode *prev=NULL, *next=head->next;
    while(head)
    {
        head->next = prev;
        prev = head;
        head = next;
        next = head->next;
    }
     return prev;
}
```



图（1）初始状态

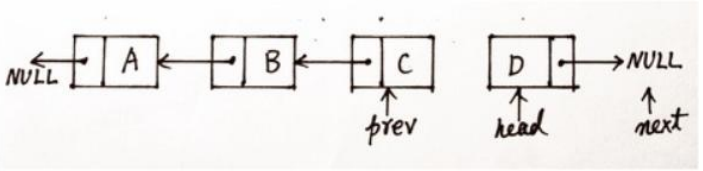


图（2）经过第一次迭代后的状态



图（3）经过第二次迭代后的状态

循环终止条件呢？现在对图（3）的状态再迭代一次得到图（4）的状态：



图（4）经过第三次迭代后的状态

伴随指针的思路

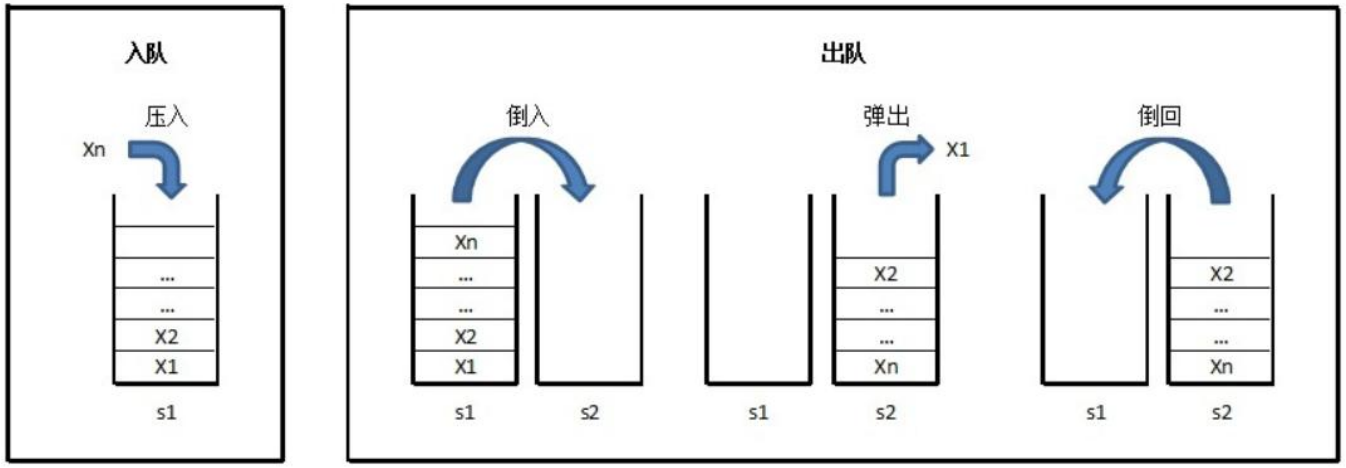
- 已知单链表L，请返回其中间节点q（如节点数为偶数，则取当中两个中的前一个）
- 已知单链表L，请查找链表中倒数第k个位置上的节点。若查找成功，输出数据域上的值，并返回1，否则返回0。
- 设单链表L=(a_1,a_2,...a_{n-1},a_n)，请设计一个空间复杂度为O(1)且时间上尽可能高效的算法，重新排列L为L′ = (a_1, a_n, a_2, a_{n-1}, ...)。

栈实现队列

```
public void push(int element) {
    // write your code here
    stack1.push(element);
}


public int pop() {
    // write your code here
   if(stack2.empty())
   while(!stack1.empty()){
       int s1=stack1.pop();
       stack2.push(s1);
   }


  return stack2.pop();
}
```



栈实现队列

- 一个栈实现队列，可以用递归，本质是利用系统的栈

```
class MyQueue {
private:
  stack<int> stk;
public:
  void enqueue(int x) {
    stk.push(x);
  }

  bool dequeue(int &x) {
    if (stk.empty()) return false;

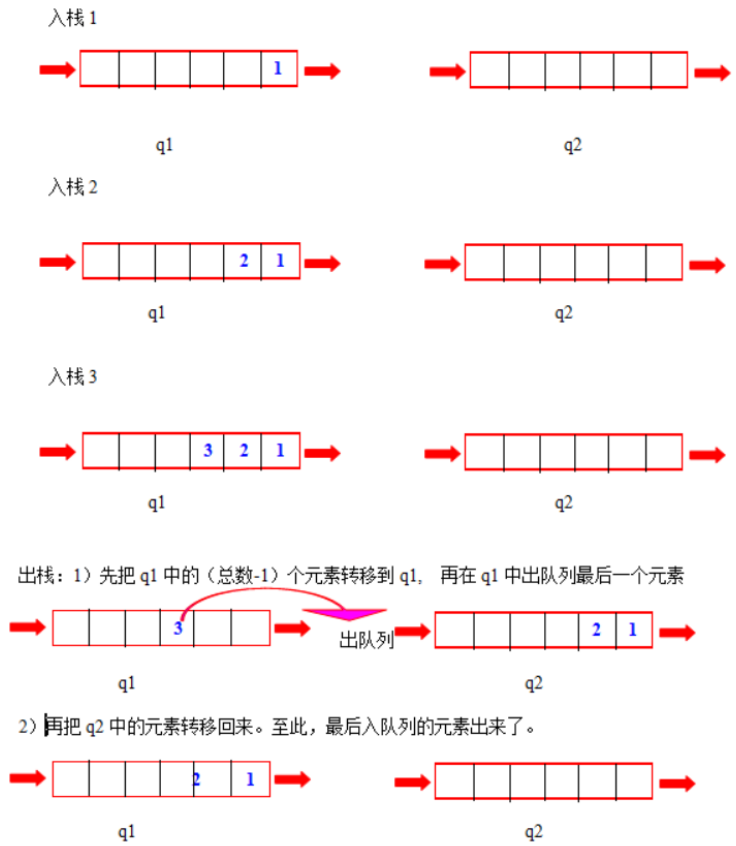
    int ele = stk.top(); stk.pop();
    // if we only want to check the front value, we need to add "stk.push(ele)"
    // after the "x=ele"
    if (!dequeue(x)) x = ele;
    else stk.push(ele);

    return true;
  }

};
```

队列实现栈

- 两个队列实现一个栈
- 一个队列实现一个栈



数制转换

- 请写出10进制到4进制转换的代码

```
void conversion(){
    InitStack();
    scanf(“%d”,N);
    while(N){
      push(S,N%4);
      N = N/4;
    }
    while(!StackEmpty(S)){
      pop(S.e);
      printf(“%d”,e);
    }
}
```

二叉树层次序遍历的实现

算法思想：
①初始化一个辅助队列
②根结点入队
③若队列非空，则队头结点出队，访问该结点，并将其左、右孩子插入队尾（如果有的话）
④重复③直至队列为空

```
//层序遍历
void LevelOrder(BiTree T){
    LinkQueue Q;
    InitQueue(Q);                   //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);                   //将根结点入队
    while(!IsEmpty(Q)){             //队列不空则循环
        DeQueue(Q, p);              //队头结点出队
        visit(p);                   //访问出队结点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild);   //左孩子入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild);   //右孩子入队
    }
}
```

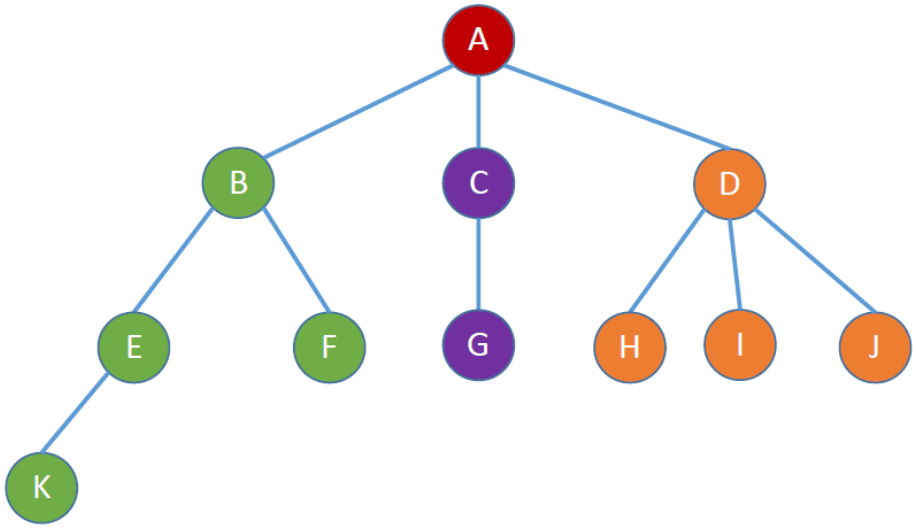
```
//二叉树的结点（链式存储）
typedef struct BiTNode{
    char data;
    struct BiTNode  *lchild,*rchild;
}BiTNode,*BiTree;

//链式队列结点
typedef struct LinkNode{
    BiTNode * data;
    struct LinkNode *next;
}LinkNode;

typedef struct{
    LinkNode *front,*rear; //队头队尾
}LinkQueue;
```

存指针而不是结点

树的层次序遍历



广度优先遍历

3）层次遍历（用队列实现）

①若树非空，则根节点入队
②若队列非空，队头元素出队并访问，同时将该元素的孩子依次入队
③重复②直到队列为空



哈夫曼树

【2014 统考真题】二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T，采用二叉链表存储，结点结构为

left	weight	right

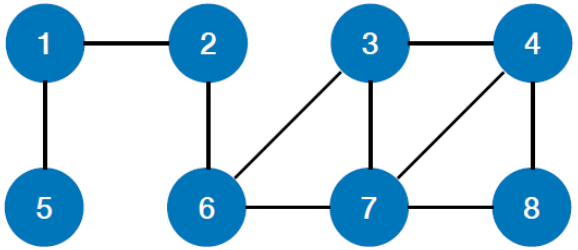
其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

1）给出算法的基本设计思想。

2）使用 C 或 C++语言，给出二叉树结点的数据类型定义。

3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

图的广度优先遍历代码实现



广度优先遍历（Breadth-First-Search, BFS）要点：

1. 找到与一个顶点相邻的所有顶点
2. 标记哪些顶点被访问过
3. 需要一个辅助队列

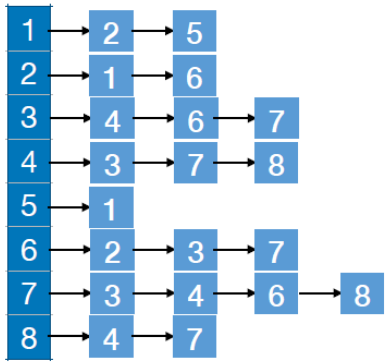
•FirstNeighbor(G,x)：求图G中顶点x的第一个邻接点，若有则返回顶点号。若x没有邻接点或图中不存在x，则返回-1。

•NextNeighbor(G,x,y)：假设图G中顶点y是顶点x的一个邻接点，返回除y之外顶点x的下一个邻接点的顶点号，若y是x的最后一个邻接点，则返回-1。

```
bool visited[MAX_VERTEX_NUM];    //访问标记数组
```

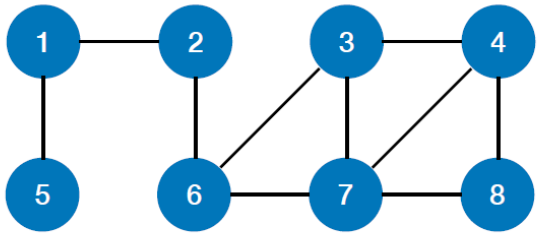
	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	0	1	0	0	1	0

邻接矩阵



邻接表

图的广度优先遍历代码实现



初始都为false

```
bool visited[MAX_VERTEX_NUM];    //访问标记数组

//广度优先遍历
void BFS(Graph G,int v){     //从顶点v出发，广度优先遍历图G
    visit(v);                       //访问初始顶点v
    visited[v]=TRUE;                //对v做已访问标记
    Enqueue(Q,v);                   //顶点v入队列Q
    while(!isEmpty(Q)){
        DeQueue(Q,v);               //顶点v出队列
        for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
            //检测v所有邻接点
            if(!visited[w]){        //w为v的尚未访问的邻接顶点
                visit(w);           //访问顶点w
                visited[w]=TRUE;//对w做已访问标记
                EnQueue(Q,w);       //顶点w入队列
            }//if
    }//while
}
```

	1	2	3	4	5	6	7	8
visited	false	false	false	false	false	false	false	false

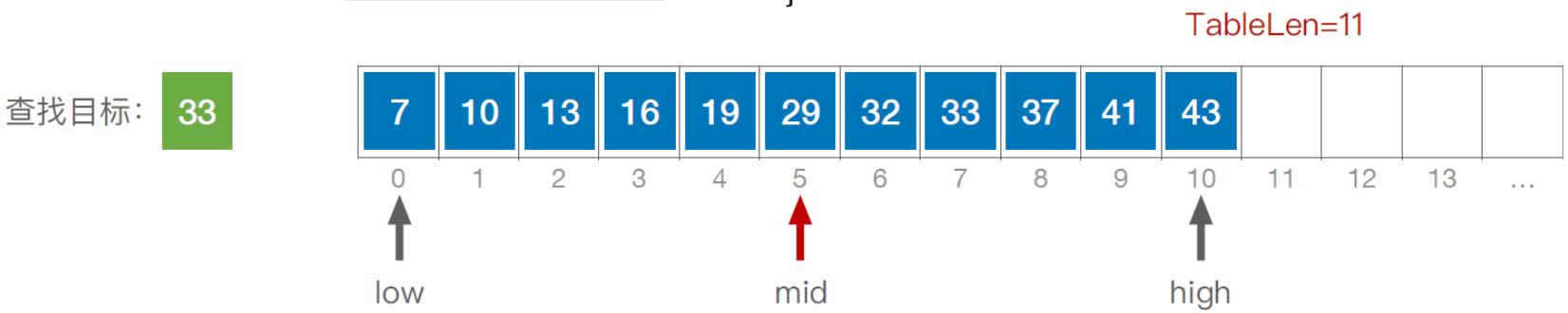
折半查找的实现

```
typedef struct{          //查找表的数据结构（顺序表）
    ElemType *elem;      //动态数组基址
    int TableLen;        //表的长度
}SSTable;
```

折半查找，又称“二分查找”，
仅适用于有序的顺序表。

顺序表拥有随机访问的特性，链表没有

```
//折半查找
int Binary_Search(SSTable L,ElemType key){
    int low=0,high=L.TableLen-1,mid;
    while(low<=high){
        mid=(low+high)/2;            //取中间位置
        if(L.elem[mid]==key)
            return  mid;             //查找成功则返回所在位置
        else if(L.elem[mid]>key)
            high=mid-1;              //从前半部分继续查找
        else
            low=mid+1;               //从后半部分继续查找
    }
    return  -1;                      //查找失败，返回-1
}
```



插入排序实现

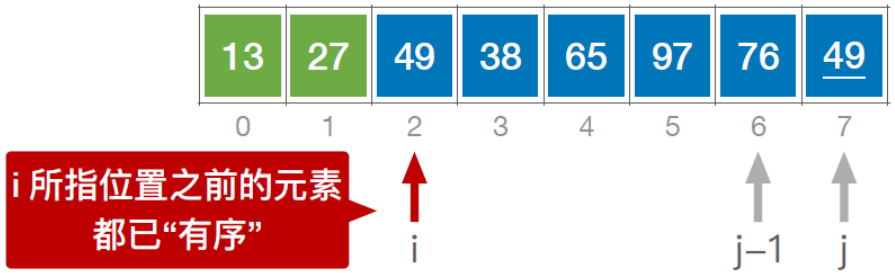
```
//直接插入排序
void InsertSort(int A[],int n){
    int i,j,temp;
    for(i=1;i<n;i++)                    //将各元素插入已排好序的序列中
        if(A[i]<A[i-1]){                //若A[i]关键字小于前驱
            temp=A[i];                  //用temp暂存A[i]
            for(j=i-1;j>=0 && A[j]>temp;--j)    //检查所有前面已排好序的元素
                A[j+1]=A[j];            //所有大于temp的元素都向后挪位
            A[j+1]=temp;                //复制到插入位置
        }
}
```

冒泡排序实现

```
//交换
void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}
```

```
//冒泡排序
void BubbleSort(int A[],int n){
    for(int i=0;i<n-1;i++){
        bool flag=false;                //表示本趟冒泡是否发生交换的标志
        for(int j=n-1;j>i;j--)          //一趟冒泡过程
            if(A[j-1]>A[j]){            //若为逆序
                swap(A[j-1],A[j]);      //交换
                flag=true;
            }
        if(flag==false)
            return;                     //本趟遍历后没有发生交换，说明表已经有序
    }
}
```

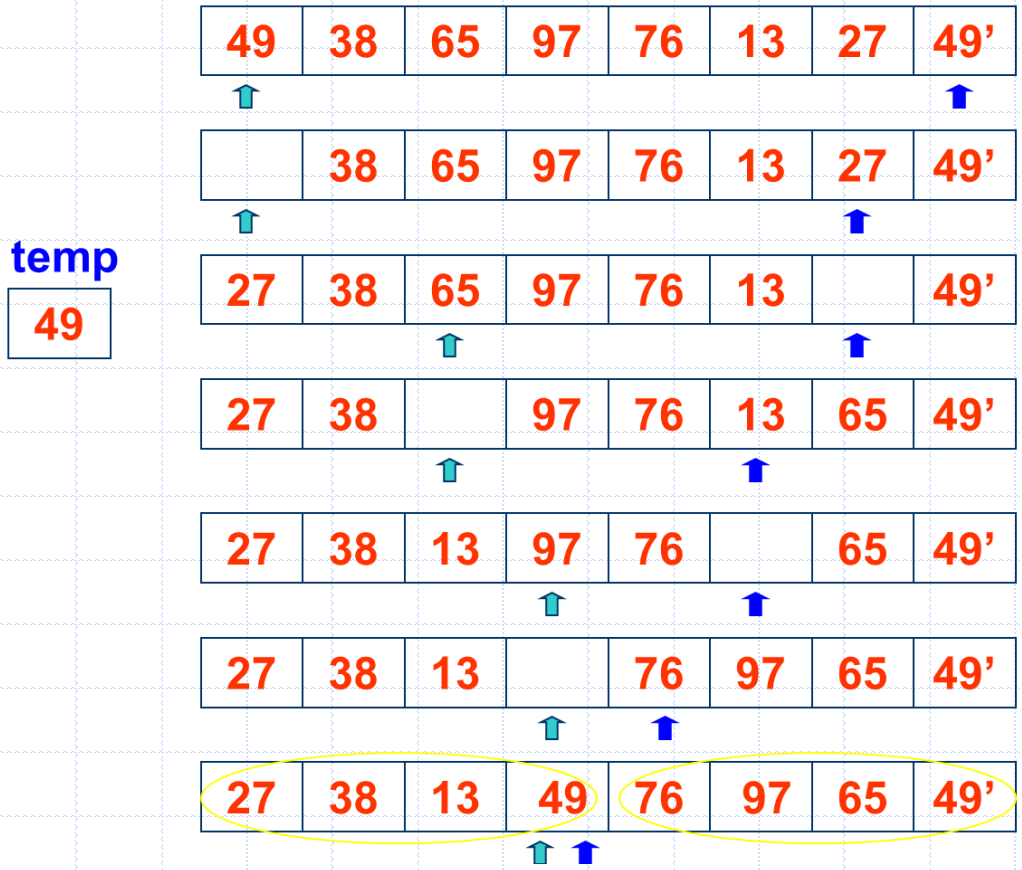
只有A[j-1]>A[j]时才交换，因此算法是稳定的



i 所指位置之前的元素都已“有序”

快速排序

- 口诀：
  前面有空格，从后往前看
  后面有空格，从前往后看



快速排序的实现

```
//用第一个元素将待排序序列划分成左右两个部分
int Partition(int A[],int low,int high){
    int pivot=A[low];          //第一个元素作为枢轴
    while(low<high){           //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];        //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];        //比枢轴大的元素移动到右端
    }
    A[low]=pivot;              //枢轴元素存放到最终位置
    return  low;               //返回存放枢轴的最终位置
}
```

```
93  //快速排序
94  void QuickSort(int A[],int low,int high){
95      if(low<high){   //递归跳出的条件
96          int pivotpos=Partition(A,low,high); //划分
97          QuickSort(A,low,pivotpos-1);   //划分左子表
98          QuickSort(A,pivotpos+1,high);  //划分右子表
99      }
100 }
```

快速排序

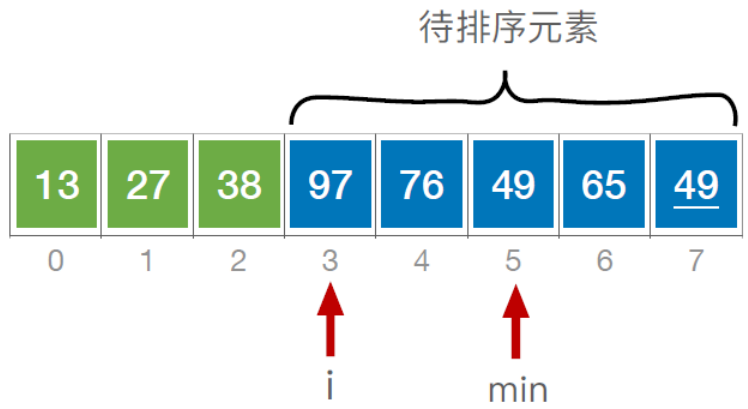
- 试编写一个算法，使之能够在数组L[1...n]中找出第k小的元素
  - 排序
  - 堆
  - 快排

快速排序

- 已知由n(n>=2)个正整数构成的集合$A = \{a_k | 0<=k<n\}$，将其划分为两个不相交的子集A_1和A_2，元素个数分别是n_1和n_2，A_1和A_2中的元素之和分别为S_1和S_2。试设计一个尽可能高效的划分算法，满足$|n_1-n_2|$最小且$|S_1-S_2|$最大，并说明所设计算法的时间复杂度和空间复杂度。

直接选择排序实现

```
//简单选择排序
void SelectSort(int A[],int n){
    for(int i=0;i<n-1;i++){                 //一共进行n-1趟
        int min=i;                          //记录最小元素位置
        for(int j=i+1;j<n;j++)              //在A[i…n-1]中选择最小的元素
            if(A[j]<A[min]) min=j;          //更新最小元素位置
        if(min!=i) swap(A[i],A[min]);       //封装的swap()函数共移动元素3次
    }
}
```




```
//交换
void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}
```

堆调整算法

```
void HeapifyUp(Node node){
    if(node==null)   return;
    int x = label(node);                                    //取得node节点的值
    //当不为根节点且父节点小时循环
   while(parent(node)!=null&&label(parent(node))<x)
          setLabel(node, label(parent(node)));              //将父节点值置给当前节点
          node=parent(node);                                //指针指向父节点
    }
    setLabel(node,x);                                       //完成一个节点的调整
}
void heapify(Node[ ] tree)
{   for(int j=0;j<tree.length;j++)
          HeapifyUp(Node[j]);
}
```

归并排序

- 若在待排序的数据中，奇数位的数据是排好序的，偶数位的数据也是排好序的，请给出一种排序方法的思路，使得速度最快，并说明理由。

24期中回顾

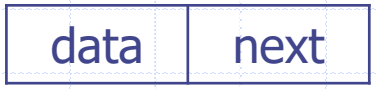
时间复杂度

- 设n是描述问题规模的正整数，则下列程序段的时间复杂度是（ ）。

```
i = n*n;
while(i!=1)
        i=i/2;
```

A. O(logn)
B. O(n)
C. $O(n^{1/2})$
D. $O(n^2)$

链表

- 已知头指针h指向一个带头结点的非空单循环链表L，结点结构为 

，其中next是指向直接后继结点的指针，p是尾指针，q是临时指针。现在要删除该链表的第一个元素，正确的语句序列是（）。

A. h->next=h->next->next; q=h->next; free(q);
B. q=h->next; h->next=h->next->next; free(q);
C. q=h->next; h->next=q->next; if(p!=q) p=h; free(q);
D. q=h->next; h->next=q->next; if(p==q) p=h; free(q);

链表

- 静态链表结点的类型定义如下，假设S表示静态链表的数组。

```
typedef struct node{
        DataType data;
        int      link;
}SListNode;
```

若逻辑上第k个结点的下标是i，则逻辑上第k+1个结点的数据是（）。

A. S[i+1].data
B. S[k+1].link.data
C. S[S[i].link].data
D. S[S[k].link].data

栈

- 现有一个共享栈S，其低位是栈S1，高位是栈S2，低位栈的栈顶地址从低向高增长，高位栈的栈顶地址从高到低减小，要求该共享栈在一端非空时，栈指针指向当前元素的下一位置，则在下列说法中，（）是错误的。

A. 该共享栈降低了溢出的可能性，提高了空间利用率

B. 当两端都非空且S1.top+1=S2.top时，共享栈满

C. S1的入栈操作是S[S1.top++]=x，出栈操作是x=S[--S1.top]

D. S2的入栈操作是S[S2.top--]=x，出栈操作是x=S[++S2.top]

矩阵的存储

- 假设一个1000*850的稀疏矩阵有1000个非零元素，每个整数占2B，每个矩阵元素占4B，则用三元组存储该矩阵时所需字节数是（）。
  - A. 4000
  - B. 7000
  - C. 8000
  - D. 18000

矩阵的存储

- 二维数组A按行优先方式存储，每个元素占用1个存储单元，若元素A[0][0]的存储地址是100，A[3][3]的存储地址是220，则元素A[5][5]的存储地址是（）。
  - A. 295
  - B. 300
  - C. 301
  - D. 306

KMP算法

- KMP 算法使用修正后的 next 数组进行模式匹配，模式串 S="aabaab"，当主串中某字符与 S 中某字符失匹时，S 向右滑动的最长距离是（）。
  A. 5
  B. 4
  C. 3
  D. 2

树

- 在二叉树的顺序存储中，每个结点的存储位置与其双亲、左右孩子结点的位置都存在一个简单映射关系，因此可与三叉链表对应。若某二叉树共有n个结点，采用三叉链表存储时，每个结点的数据域占用d字节，每个指针域占用4B，采用顺序存储，且最后一个结点的下标为k（起始下标为1），那么当（）时采用顺序存储更节省空间。

A. d < 12n/(k-n)

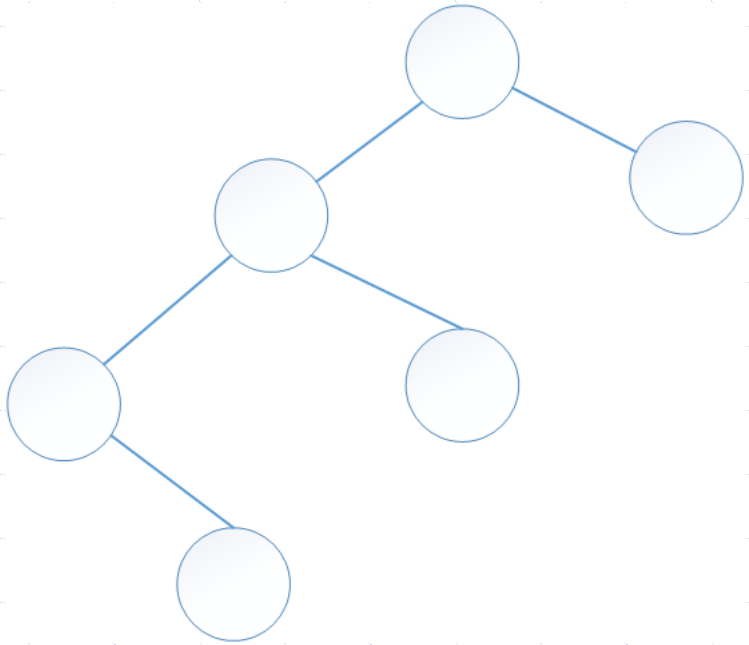
B. d > 12n/(k-n)

C. d < 12n/(k+n)

D. d > 12n/(k+n)

二叉树的遍历

- 已知一棵二叉树的树形如图所示，若其后序遍历序列为fdbeca，则其前序遍历序列是（）。
  A. aedfbc
  B. acebdf
  C. cabefd
  D. dfebac



森林与二叉树

- 某森林F对应的二叉树为T，若T的前序遍历序列是abdcegf，中序遍历序列是bdaegcf，则F中树的棵数是（）。
  - A. 1
  - B. 2
  - C. 3
  - D. 4

并查集

- 下列关于并查集的叙述中，（）是错误的。
  - A. 并查集是用双亲表示法存储的树
  - B. 并查集可以用于实现Kruskal算法
  - C. 并查集可以用于判断无向图的连通性
  - D. 在长度为n的并查集中进行查找操作的时间复杂度为O(logn)

哈夫曼树

- 在一次哈夫曼编码中，若要求编码长度小于等于4，假设现在已对两个字符编码为0和10，则最多还可以为（）个字符进行编码。

A. 3
B. 4
C. 5
D. 6

图

- 若G是一个有36条边的非连通简单无向图，则图G的顶点数至少是（）。
  - A. 11
  - B. 10
  - C. 9
  - D. 8

图

- 设图的邻接矩阵A如右图所示，各顶点的度依次是（）。

A. 1，2，1，2

B. 2，2，1，1

C. 3，4，2，3

D. 4，4，2，2

$$A=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

图

- 有向图G有1,000个顶点和2,000条边，使用邻接表进行存储，假设每个顶点需要2个字节保存，每条边需要3个字节保存，则最接近其需要的存储空间是（）。
  - A. 1,006,000个字节
  - B. 14,000个字节
  - C. 12,000个字节
  - D. 8,000个字节

填空题

- 后缀表达式xy+10/ab*c/+xy-ab-*-对应的中缀表达式为_____。

- (x+y)/10+a*b/c-(x-y)*(a-b)

填空题

- 按教材中循环队列的实现方式，设循环队列的容量为50（序号从0到49），现经过一系列的入队和出队运算后，有front =16，rear=5（rear指向队尾元素的后一位置），则当前循环队列中元素个数为_____。

- 39

填空题

- 已知数组A[1..10, 1..10]为对称矩阵，其中每个元素占5个单元。现将其下三角部分按行优先次序存储在起始地址为1000的连续内存单元中，则地址1095中存放的元素在数组A中的位置是_____。

- A[5][6] 或 A[6][5]

填空题

- 广义表GL=((a, b, c),(d, e, f))，假设求表头操作为head，求表尾操作为tail，则head[tail[tail [head[tail[GL]]]]]=_____。

- f

填空题

- 已知完全二叉树的第7层有10个叶结点，规定根结点层次为1，则整棵二叉树的结点数最多为_____。

- $2^7-1+(2^6-10)*2 = 235$

问答题

- 森林的后根遍历结果为DECBAFIKLG，先根遍历结果为ADCEBIFGKL。要求：
  - 画出这个森林对应的二叉树；
  - 画出这棵二叉树经过后序线索化后树上的线索指针。

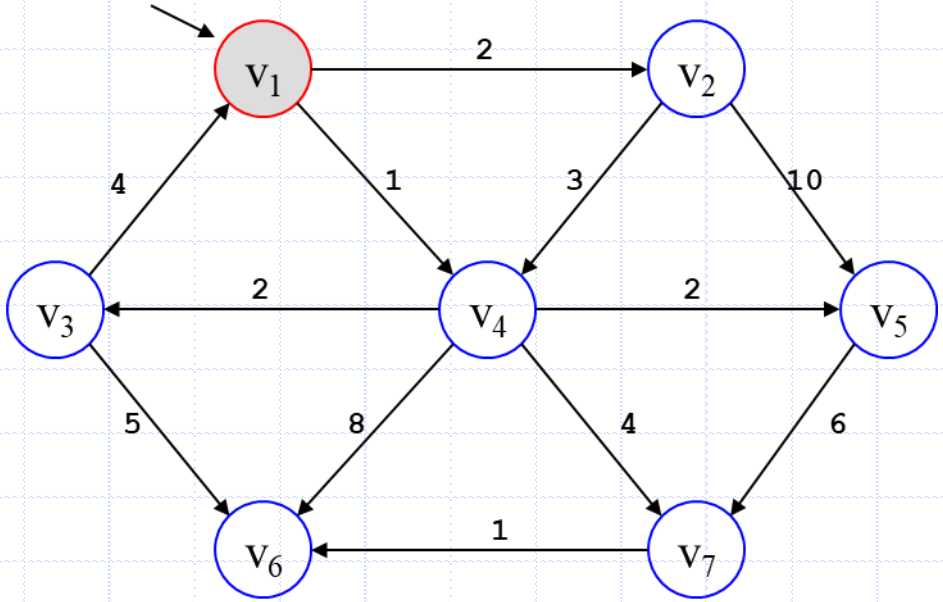
问答题

- 一带权无向图的邻接矩阵如下图所示，试分别给出Prim和Kruskal生成最小生成树的构造过程。

$$\begin{vmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 2 & 0 & 0 \\ 1 & 1 & 0 & 0 & 3 & 0 \\ 0 & 2 & 0 & 0 & 1 & 1 \\ 0 & 0 & 3 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{vmatrix}$$

问答题

- 已知有向图G如下图所示，请使用Dijkstra算法求出顶点v1到其他各顶点的最短路径长度（需有必要的中间步骤）。



代码题

- 输入n(n<=6)，请输出1至n数字组成的n位数字（不可重复）

```
int a[6];
int visited[6];

//递归输出，输出m列，每列允许输出1-n
int DFS(int m,int n)
{
    int i;
    if(m==0)
    {
        for(i=0;i<n;i++)
            cout<<a[i]<<" ";
        cout<<endl;
        return 0;
    }
    for(i=1;i<=n;i++)
    {
      if(!visited[i-1]){
        a[n-m]=i;
        visited[i-1] = 1;
        DFS(m-1,n);
        visited[i-1] = 0;
    }

}
```

代码题

- 给出有向图的邻接表类定义，并定义其上的广度优先搜索方法。类定义可适当简化，可省略与所定义的广度优先搜索方法无关的属性和方法的定义。

算法从顶点v开始实现非递归深度优先遍历。

```
class AdjList{
    void Traverse(){
        for(vi=1;vi<=n;vi++)
            if(!visited[vi])Traver(g,vi);
    }
    void Traver(vertype v){
        struct arc *stack[];
        visited[v]=1; printf(v);  //输出顶点v
        top=0;  p=node[v]->next;  stack[++top]=p;
        while(top>0||p!=null){
            while(p)
            {
                if(p&&visited[p->adjvex])p=p->next;
                else{
                        printf(p->adjvex);  visited[p->adjvex]=1;
                        stack[++top]=p;  p=node[p->adjvex]->next;
                }//else
                if(top>0){p=stack[top--];  p=p->next;}
            }//while
        }//while
    }//算法结束。
}//class定义结束
```

Q & A